

Platform Implementation Plan

Version 3.2 | June 2026 Confidential — Build Specification for Claude Code CLI (v3.2 adds §19.4 — Operating Account Creation (E7) — and Section 21 — Post-Pilot Improvement Backlog (B1-B9, Planned). v3.1 added Section 19 — Admin Panel Enhancements and Section 20 — Deployment Runbook, on top of the completed v3.0 Phase 1-3 build.)

||| | --- | --- | | Client | Haleelo Tower — Hargeisa, Somaliland | | Platform | Custom-built Web Platform (replaces ODOO) | | Discovery Rounds | 3 rounds — 75 confirmed requirements | | Finance Officer Review | June 2026 | | Accountant Review | June 2026 | | Currency | USD only | | Language | English only | | Hosting | Client VPS (Ubuntu 22.04) |

SECTION 1 — EXECUTIVE SUMMARY

Haleelo Tower is replacing ODOO with a purpose-built, multi-tenant commercial building management platform. This v3.0 specification is the final confirmed build document incorporating three rounds of client discovery (75 confirmed requirements), the Finance Officer questionnaire (June 2026), and the Accountant review session (June 2026).

Platform Purpose

The platform manages every operational aspect of Haleelo Tower — a multi-floor commercial building in Mogadishu, Somalia — including conference hall booking, long-term office and educational space leasing, full double-entry accounting, HR and payroll, tenant communications, and a public-facing marketing website.

Key Changes from v1.1

- Conference hall booking now follows a 4-step approval chain: Client submits → Operations/Admin creates → Admin approves → Finance (Accountant) gives final approval
- Full professional accounting module aligned with double-entry accounting standards (chart of accounts, journal entries, trial balance, P&L, balance sheet)
- Catering packages (Silver / Gold / Platinum) with separate DJ and Cameraman add-on pricing
- Product management system for all building offerings (spaces, services, add-ons)
- 5 separate operating accounts tracked: Edahab×2 (conference halls & office rentals), ZAAD×2 (conference halls & office rentals), Darasalam Bank
- Bill code + account code system for every financial transaction
- Electricity meter-to-invoice management with rate history
- Vendor purchase orders and bills module
- Email provider changed to Resend API (resend/resend-php on Laravel)
- Brand identity: Poppins typeface, #1B2D4F navy + #C9A052 gold color palette
- Late payment penalties and bank reconciliation tool moved to Future Phase per client instruction

Subdomains

| Subdomain | Purpose | Access | Notes | | --- | --- | --- | --- | | halelotower.so | Public Website | Next.js 14, publicly accessible, no login required | | portal.halelotower.so | Tenant Portal | Next.js 14, authenticated tenants only | | admin.halelotower.so | Admin Dashboard | Next.js 14, internal staff only, RBAC | | api.halelotower.so | Laravel REST API | Backend, not publicly browseable | |

Platform Users

| Role / User Type | Count | Scope | | --- | --- | --- | | Super Admin | 1 | Full system access, settings, audit logs, user management | | Admin | 1 | Day-to-day operations, first approver on conference bookings, tenant management | | Operations | 2 | Booking intake, tenant onboarding, invoice generation | | Finance | 1 | All financial operations, final approver on conference bookings | | Long-Term Tenant | Varies | Tenant portal access — invoices, documents, maintenance | | General Public | Unlimited | Public website only — no account required |

SECTION 2 — BUILDING & SPACE OVERVIEW

Haleelo Tower is a multi-floor commercial building located in Mogadishu, Somalia. The building offers three categories of rentable space: educational facilities, private offices, and conference/event halls.

Floor Layout

| Floor | Space Type | Details | | --- | --- | --- | | Basement | Educational Facility | Entire floor leased to one university tenant. Furnished with projectors, whiteboards, lab equipment. Billed per semester. | | Ground Floor | Offices (×10) | Ten individual rentable office rooms. Each independently bookable. Billed monthly. | | 1st Floor | Offices (×3) + Hall A | Three private offices + Hall A (50-seat conference hall). Hall A bookable per session or custom duration. | | 2nd Floor | Offices (×2) + Hall B | Two private offices + Hall B (100-seat conference hall). Hall B bookable per session or custom duration. | | 3rd Floor | Hall C (Main Hall) | Hall C — the primary event space with 500-seat capacity. Used for large corporate events, conferences, ceremonies. |

Conference Hall Session Windows

All three conference halls (A, B, C) share the same standard session time structure. Session times are configurable in System Settings.

| Session | Start | End | Notes | | --- | --- | --- | --- | | Morning | 08:00 | 13:00 | 5 hours | | Afternoon | 15:00 | 18:30 | 3.5 hours | | Evening | 19:00 | 23:00 | 4 hours | | Custom | Configurable | Configurable | Minimum 1 hour, maximum 1 month. Bookable up to 3 months in advance. |

Space Categories

Conference Halls (A, B, C) — Bookable per session (Morning/Afternoon/Evening) or custom duration. Subject to 4-step approval chain. Optional catering packages (Silver/Gold/Platinum), DJ, Cameraman. Priced per session or per hour. Office Spaces (Ground, 1st, 2nd Floor) — Leased monthly. Tenant onboarding required. KYC and lease document collection. Electricity billed separately via meter readings. Educational Facility (Basement) — Leased per semester to a single university tenant. Includes furnished space with lab equipment.

SECTION 3 — PLATFORM ARCHITECTURE

The platform is composed of four distinct applications sharing a single PostgreSQL database accessed exclusively through the Laravel REST API. No application other than the Laravel backend ever connects directly to the database.

Application Separation of Concerns

halelotower.so (Public Website) Next.js 14 App Router. Publicly accessible — no authentication. Serves SEO-optimised marketing pages, space listings, and the booking request form. Static generation with ISR for availability data. Calls the Laravel API only for: booking form submission, availability checks. portal.halelotower.so (Tenant Portal) Next.js 14 App Router. Authenticated routes for tenant users only. Tenants log in with email + password (no 2FA). HTTP-only cookie token managed by Laravel Sanctum. Renders

invoices, lease documents, maintenance requests, announcements. admin.halelotower.so (Admin Dashboard) Next.js 14 App Router. Authenticated routes for internal staff (Super Admin, Admin, Operations, Finance). 2FA required on every login. Role-based middleware on every route — components conditionally rendered based on permissions array returned by /api/auth/me. Full system management UI. api.halelotower.so (Laravel API) Laravel 11 REST API. Not publicly accessible as a browseable URL. All business logic lives here. Issues Sanctum SPA tokens. CORS configured to allow only the three subdomain origins. Every route protected by Sanctum + Spatie permission middleware.

Database Schema Overview

All tables use PostgreSQL 16. Money columns are decimal(15,2). Soft deletes (deleted_at) on all core tables except audit_logs, journal_entries, and journal_entry_lines which are immutable. Every table has created_at and updated_at timestamps.

Identity & Access

- users — id, name, email, password, role, status, phone, profile_photo_url, 2fa_secret, created_at
- roles — id, name, guard_name (Spatie)
- permissions — id, name, guard_name (Spatie)
- role_user, role_has_permissions, model_has_permissions (Spatie pivot tables)

Building & Products

- floors — id, name, level, description
- spaces / products — id, name, slug, type, floor_id, capacity, description, amenities (JSON), base_price, price_unit, photos (JSON), status
- product_services — id, product_id, service_name, service_type, price, active
- catering_packages — id, name, slug, description, base_price, active
- catering_package_items — id, package_id, service_name, description

Bookings

- bookings — id, booking_code, type, product_id, client fields, session_type, booking_date, start_time, end_time, recurring, recurrence_rule (JSON), catering_package_id, dj_requested, cameraman_requested, pricing fields, status, payment_status, notes, created_by
- booking_services — id, booking_id, service_name, price
- booking_status_logs — id, booking_id, from_status, to_status, changed_by_user_id, notes, created_at
- waiting_list — id, product_id, session_type, booking_date, client_name, client_email, client_phone, created_at

Tenants & Leases

- tenants — id, company_name, contact_person_name, email, phone, national_id, type, status, portal_access, portal_password_hash
- tenant_documents — id, tenant_id, document_type, file_url, uploaded_at
- leases — id, tenant_id, space_id, lease_code, start_date, end_date, monthly_rent, billing_cycle, status, security_deposit_amount, security_deposit_status, contract_file_url
- security_deposits — id, lease_id, tenant_id, amount, status, received_date, returned_date, notes

Finance — Invoicing

- invoices — id, invoice_code, type, tenant_id, lease_id, booking_id, issue_date, due_date, billing_period, lpo_number, subtotal, total_amount, status, payment_account_id
- invoice_line_items — id, invoice_id, description, quantity, unit_price, line_total, account_code_id

- payments — id, payment_code, type, invoice_id, vendor_bill_id, amount, payment_date, payment_method, account_id, reference_number

Finance — Vendors & Expenses

- vendors — id, name, contact_person, phone, email, category, notes, active
- purchase_orders — id, po_code, vendor_id, order_date, expected_delivery_date, status, total_estimated_amount
- purchase_order_items — id, po_id, description, quantity, estimated_unit_price, line_total
- vendor_bills — id, bill_code, vendor_id, po_id, bill_date, due_date, status, total_amount, expense_account_id, payment_account_id, receipt_file_url
- vendor_bill_items — id, bill_id, description, quantity, unit_price, line_total, expense_account_id
- expenses — id, expense_code, description, amount, expense_date, expense_account_id, payment_account_id, booking_id, receipt_file_url

Finance — Accounting Engine

- accounts — id, chart_of_account_id, name, type, account_identifier, active
- account_transactions — id, account_id, type, amount, description, reference_code, journal_entry_id, transaction_date
- chart_of_accounts — id, code, name, type, parent_id, description, active
- journal_entries — id, journal_code, entry_date, description, reference_code, posted_by_user_id, source
- journal_entry_lines — id, journal_entry_id, account_id, type (debit/credit), amount, description
- electricity_rates — id, rate_per_kwh, effective_from, effective_to, created_by
- electricity_readings — id, electricity_bill_code, tenant_id, space_id, reading_date, billing_period_month, previous_reading, current_reading, kwh_consumed, rate_id, rate_per_kwh (snapshot), total_charge, invoice_id, status
- reference_sequences — id, prefix, year, last_sequence

HR & Payroll

- employees — id, employee_code, full_name, job_title, department, employment_type, base_salary, daily_rate, start_date, status, contract_file_url
- attendance_logs — id, employee_id, month, working_days_in_month, days_worked, days_absent, late_arrivals
- leave_requests — id, employee_id, leave_type, start_date, end_date, days_count, status, approved_by
- overtime_records — id, employee_id, date, hours, rate_multiplier, total_amount
- deductions — id, employee_id, payroll_run_id, type, amount, description
- payroll_runs — id, run_code, month, department_filter, total_gross, total_deductions, total_net, status
- payslips — id, payslip_code, payroll_run_id, employee_id, gross_pay, overtime_pay, total_deductions, net_pay, pdf_file_url, sent_via_whatsapp, sent_via_email

Operations

- maintenance_requests — id, request_code, tenant_id, space_id, category, description, photo_url, status, assigned_to_user_id, notes, resolved_at
- announcements — id, subject, body, sent_to, sent_via (JSON), sent_by_user_id, sent_at
- tenant_notifications — id, tenant_id, message, type, read, link, created_at
- audit_logs — id, user_id, user_name, user_role, action, model_type, model_id, old_values (JSON), new_values (JSON), ip_address, created_at
- system_settings — id, key, value, description, updated_by

SECTION 4 — USER ROLES & PERMISSIONS

4.1 Role Definitions

Super Admin — IT / System Administrator (1 person)

- Full unrestricted access to all system areas
- System settings, chart of accounts, account codes
- User management: create, edit, deactivate any account
- Audit log access (read-only, cannot delete)
- Can reset any user's password
- Receives automated monthly financial report
- Admin — Building Manager (1 person)
- Day-to-day operational management
- First approver on conference hall bookings (Admin Pending → Accountant Pending)
- Tenant management: onboarding, lease review
- Access to most reports (read-only for financial reports)
- Broadcast announcements to tenants
- Can create/edit products and catering packages
- Cannot access payroll or journal entry creation
- Receives automated monthly financial report
- Operations — Receptionist / Operations Staff (×2)
- Booking intake: create bookings on behalf of clients
- Tenant onboarding: document collection, data entry
- Invoice generation (create invoice records)
- No access to financial reports
- No user management
- No access to chart of accounts, journal entries, or payroll
- No approval actions (cannot approve/reject bookings)
- Finance — Finance Officer (1 person)
- Exclusive write access to all financial operations
- Final approver on conference hall bookings (Accountant Pending → Booking Approved)
- Invoice management, payment recording, vendor bills
- Chart of accounts management (add sub-accounts)
- Journal entries (manual and review of auto-posted)
- Electricity billing management
- Payroll management (full HR/payroll access)
- All financial reports with export capability
- Expense recording and vendor management

4.2 Permissions Matrix

Legend: ✓ = Full access | ✗ = No access | R = Read only

Module / Action	Super Admin	Admin	Operations	Finance	---	---	---	---	---	System Settings (General, Email, WhatsApp)
System Settings (Session Times, Payment Terms)	✓	✓	✗	✗	✗	✗	✗	✗	✗	Electricity Rate Configuration
User Management (Create/Edit/Deactivate)	✓	✗	✗	✗	✗	✗	✗	✗	✗	User Password Reset
Audit Trail (View)	✓	✓	✗	✗	✗	✗	✗	✗	✗	Audit Trail (Export)
Product Management (Create/Edit/Delete)	✓	✓	✗	✗	✗	✗	✗	✗	✗	Product Management (View)
Catering Package Configuration	✓	✓	✗	✗	✗	✗	✗	✗	✗	Conference Hall Bookings — Create
Conference Hall Bookings — Admin Approve	✓	✓	✗	✗	✗	✗	✗	✗	✗	Conference Hall Bookings — Finance Final Approve
Conference Hall Bookings — Reject	✓	✓	✗	✗	✗	✗	✗	✗	✗	Conference Hall Bookings — Cancel
Office Lease Bookings — Create	✓	✓	✓	✓	✓	✓	✓	✓	✓	Office Lease Bookings — Approve
Waiting List — View/Manage	✓	✓	✓	✓	✓	✓	✓	✓	✓	Tenant Management — Create/Edit
Tenant Management — Deactivate	✓	✓	✗	✗	✗	✗	✗	✗	✗	Tenant Portal Access (Admin view)
Invoice — Create/Edit (Draft)	✓	✓	✓	✓	✓	✓	✓	✓	✓	Invoice — Send to Tenant
Invoice — View	✓	✓	✓	✓	✓	✓	✓	✓	✓	Payment

Recording | ✓ | X | X | ✓ | | Vendor Management | ✓ | ✓ | X | ✓ | | Vendor Bills — Create/Edit | ✓ | X | X | ✓ | | Purchase Orders | ✓ | ✓ | X | ✓ | | Expense Recording | ✓ | X | X | ✓ | | Chart of Accounts — View | ✓ | R | X | ✓ | | Chart of Accounts — Add Sub-accounts | ✓ | X | X | ✓ | | Chart of Accounts — Edit Account Codes | ✓ | X | X | X | | Journal Entries — View | ✓ | R | X | ✓ | | Journal Entries — Create Manual | ✓ | X | X | ✓ | | Electricity Readings — Record | ✓ | ✓ | X | ✓ | | Electricity Billing — Generate Invoice | ✓ | X | X | ✓ | | Financial Reports — View/Export | ✓ | R | X | ✓ | | Payroll — Manage Employees | ✓ | X | X | ✓ | | Payroll — Run Payroll / Generate Payslips | ✓ | X | X | ✓ | | HR — Attendance Logging | ✓ | ✓ | X | ✓ | | HR — Leave Requests (Approve/Reject) | ✓ | ✓ | X | ✓ | | Maintenance Requests — View/Assign | ✓ | ✓ | ✓ | X | | Maintenance Requests — Resolve | ✓ | ✓ | ✓ | X | | Broadcast Announcements | ✓ | ✓ | X | X | | Account Balances — View | ✓ | R | X | ✓ | | Inter-Account Transfers | ✓ | X | X | ✓ | | Management Dashboard KPIs | ✓ | ✓ | ✓ | ✓ |

4.3 Authentication

Staff Login Flow (2FA Required)

- Step 1: Staff navigates to admin.halelotower.so/login
- Step 2: Enters email + password
- Step 3: If credentials valid: system dispatches 6-digit OTP via WhatsApp (primary) or SMS (fallback)
- Step 4: Staff enters OTP on the 2FA verification screen
- Step 5: If OTP valid and not expired (5-minute window): Sanctum issues SPA cookie token
- Step 6: Every login requires 2FA — no persistent bypass, no 'remember this device' option
- Step 7: Session timeout: 8 hours inactivity for all staff roles

Tenant Portal Login

- Email + password only — no 2FA
- Session timeout: 24 hours inactivity
- Portal password stored as hashed value in tenants.portal_password_hash

Password Policy

- Minimum 8 characters
- Must include at least one uppercase letter
- Must include at least one number
- Must include at least one symbol (!@#\$%^&*)
- Validation enforced on both frontend (real-time) and backend (API)

Account Security

- Failed login logout: 5 consecutive failed attempts → account locked for 15 minutes
- Super Admin and Admin can unlock accounts and reset passwords via User Management
- All login attempts (success/failure) recorded in audit_logs
- Laravel Sanctum manages SPA cookie tokens — HTTP-only, Secure, SameSite=Lax
- CSRF protection enabled for all state-changing requests

4.4 Audit Trail

Every state-changing action in the system is recorded in the audit_logs table. This log is append-only and cannot be edited or deleted by any role including Super Admin.

audit_logs Table Schema

Column	Type	Description
id	bigint	Primary key
user_id	bigint FK	The authenticated user who performed the action
user_name	varchar	Snapshot of user name at time of action
user_role	varchar	Snapshot of user role at time of action
action	enum	created / updated / deleted / approved / rejected / login / logout / exported
model_type	varchar	Laravel model class name (e.g. App\Models\Booking)
model_id	bigint	ID of the affected record
old_values	jsonb	State before the change (null for created actions)
new_values	jsonb	State after the change (null for deleted actions)
ip_address	inet	Client IP address
created_at	timestamp	When the action occurred (UTC)

Audit Log Access & Filtering

- Viewable by Super Admin and Admin only
- Filterable by: user (dropdown), date range (date picker), action type (multi-select), model type (multi-select)
- Search by model_id or reference code
- Exportable as PDF (DomPDF) and Excel (Maatwebsite)
- Pagination: 50 records per page
- Immutability enforced at database level: no UPDATE or DELETE permissions on audit_logs granted to the application DB user

SECTION 5 — SYSTEM SETTINGS & CENTRAL CONFIGURATION

A single unified /settings area in the admin dashboard. All settings are stored in the system_settings table as key-value pairs (key varchar, value text, description, updated_by). Super Admin has access to all settings sections. Admin has access to a defined subset.

5.1 General Settings

- building_name — Display name for the building (used in all documents)
- logo_url — S3 URL of the uploaded logo (used in PDF templates and email headers)
- contact_email — Primary contact email for public display
- contact_phone — Primary contact phone number
- address — Full building address
- timezone — Fixed to Africa/Mogadishu (EAT UTC+3). Displayed in UI, used for scheduled job timing.
- date_format — Default: DD/MM/YYYY

5.2 Electricity Rate Configuration

- Managed via electricity_rates table (not system_settings key-value)
- rate_per_kwh (decimal, USD) — the price charged to tenants per kWh consumed
- effective_from (date) — when this rate takes effect
- effective_to (date, nullable) — auto-set when a new rate is saved
- When a new rate is saved: previous rate's effective_to is set to the day before new rate's effective_from
- Rate lookup: when calculating an electricity bill, system queries the rate where effective_from <= reading_date AND (effective_to IS NULL OR effective_to >= reading_date)
- Only Super Admin and Admin can create or modify rates
- Rate history is preserved — no rate record is ever deleted

5.3 Catering Package Configuration

- Create/edit Silver, Gold, Platinum packages and any future custom packages
- Per package: name, slug, description, list of included services, base_price (USD)
- catering_packages table: id, name, slug, description, base_price, active (boolean)

- catering_package_items table: id, package_id, service_name, description
- DJ add-on price: stored as system_settings key 'addon_dj_price' (decimal string)
- Cameraman add-on price: stored as system_settings key 'addon_cameraman_price'
- Packages apply globally to all conference halls
- Only Super Admin and Admin can edit packages
- Deactivating a package (active=false) hides it from new bookings but preserves historical data

5.4 Session Time Configuration

- session_morning_start — default '08:00'
- session_morning_end — default '13:00'
- session_afternoon_start — default '15:00'
- session_afternoon_end — default '18:30'
- session_evening_start — default '19:00'
- session_evening_end — default '23:00'
- Changes take effect immediately for new bookings; existing approved bookings are not affected

5.5 Payment Terms

- invoice_due_days — number of days from invoice issue date to due date (default: 7)
- Used when auto-generating invoices to calculate due_date = issue_date + invoice_due_days

5.6 WhatsApp API Configuration

- whatsapp_provider — 'twilio' or '360dialog'
- API credentials stored in .env (WHATSAPP_API_KEY, WHATSAPP_SENDER_NUMBER)
- Settings UI shows masked key values (last 4 chars visible) — cannot be read back in plaintext via UI
- Test button: sends a test WhatsApp message to a specified number to verify configuration

5.7 Email Configuration (Resend API)

- RESEND_API_KEY stored in .env — never exposed in UI or API responses
- resend_from_name — sender display name (e.g. 'Haleelo Tower')
- resend_from_email — verified sender address on Resend (e.g. noreply@halelotower.so)
- resend_reply_to — reply-to email address
- HTML email templates: stored in resources/views/emails/ as Blade templates
- All transactional emails sent via resend/resend-php SDK
- Test button: sends a test email to specified address

5.8 Working Hours (Payroll)

- working_hours_per_day — standard hours in a workday (default: 8)
- Used in payroll calculation: overtime = hours_worked > working_hours_per_day
- working_days_per_month — used as denominator for daily-rate calculations (default: 26)

5.9 Fiscal Year Settings

- fiscal_year_start_month — integer 1-12 (default: 1 = January)

- Used to group financial report periods and define year boundaries
- Balance sheet and P&L reports use this to determine 'current year' vs 'prior year' comparatives

SECTION 6 — PHASE 1: USER MANAGEMENT, ROLES & PERMISSIONS

6.1 User Management (Admin Dashboard)

Managed at /admin/users. Super Admin and Admin can create, edit, and deactivate staff accounts. Operations and Finance users cannot access this area.

User Record Fields

| Field | Type | Notes | | --- | --- | --- | | id | bigint | Auto-increment primary key | | name | varchar(255) | Full name | | job_title | varchar(255) | Display title (e.g. Building Manager) | | email | varchar(255) unique | Login identifier | | password | varchar(255) | Bcrypt hash | | phone | varchar(20) | Used for 2FA OTP delivery | | role | string via Spatie | super_admin / admin / operations / finance | | status | enum | active / inactive | | profile_photo_url | varchar nullable | S3 URL | | two_factor_secret | varchar nullable | TOTP secret (if TOTP-based 2FA) | | two_factor_confirmed_at | timestamp nullable | When 2FA was configured | | created_at / updated_at | timestamps | Standard Laravel timestamps | | deleted_at | timestamp nullable | Soft delete — deactivated users not shown in lists but records preserved |

User Management Operations

- Create user: form with all fields, role dropdown, sends invite email via Resend with temporary password link
- Edit user: update any field. Role changes take effect immediately on next API request.
- Deactivate user: sets status=inactive and deleted_at. User cannot log in. Their audit log entries and created records remain unchanged.
- Reactivate user: clears deleted_at, sets status=active
- Reset password: generates a password reset token, sends reset link via Resend API
- User list: searchable by name/email, filterable by role and status, sortable by name/created_at
- Pagination: 25 records per page

6.2 Role Management

Roles are defined in code and seeded into the database via a DatabaseSeeder. They are not editable through the UI in the Pilot version. Role assignment is done per user via the User Management UI.

- Package: Spatie Laravel Permission (spatie/laravel-permission)
- Roles table: seeded with super_admin, admin, operations, finance
- Permissions are defined per route/controller action and assigned to roles in RoleSeeder
- Middleware: every API route uses ->middleware(['auth:sanctum', 'role:admin|super_admin']) or ->middleware(['auth:sanctum', 'permission:manage-invoices']) pattern
- Frontend: /api/auth/me endpoint returns { user, role, permissions[] }. React context stores this. Components use a usePermission() hook to conditionally render.
- Permission check failure: API returns 403 JSON response { error: 'Forbidden', message: 'You do not have permission to perform this action' }

SECTION 7 — PHASE 2: BOOKING SYSTEM

7.1 Product Management

Every bookable space in the building is a Product. Products are the central entity that bookings, invoices, and availability logic reference.

Product Types

| Product Type | Description | | --- | --- | | conference_hall | Booked per session (Morning/Afternoon/Evening) or custom duration. Has capacity. Supports catering packages and add-ons. | | office_space | Leased monthly. Assigned to a tenant. Has electricity metering. Supports cleaning and internet add-ons. | | educational_space | Leased per semester. Single tenant. Furnished with lab equipment. |

spaces / products Table

| Field | Type | Notes | | --- | --- | --- | | id | bigint | PK | | name | varchar(255) | Display name (e.g. 'Hall C — Grand Ballroom') | | slug | varchar(255) unique | URL slug (e.g. 'hall-c-grand-ballroom') | | type | enum | conference_hall / office_space / educational_space | | floor_id | bigint FK | References floors table | | capacity | integer nullable | Max persons (for halls) | | description | text | Long-form description for public website | | amenities | jsonb | Array of strings: ['projector','AC','WiFi','whiteboard','AV system','stage'] | | base_price | decimal(15,2) | Base price per unit (session or month) | | price_unit | enum | per_session / per_month / per_semester | | photos | jsonb | Ordered array of S3 URLs | | status | enum | active / inactive | | created_at / updated_at | timestamps | | deleted_at | timestamp nullable | Soft delete |

product_services Table (Add-ons)

| Field | Type | Notes | | --- | --- | --- | | id | bigint | PK | | product_id | bigint FK | Parent product | | service_name | varchar | e.g. 'Silver Catering Package', 'DJ', 'Monthly Cleaning' | | service_type | enum | catering_package / dj / cameraman / cleaning / internet / other | | price | decimal(15,2) | USD price | | active | boolean | Whether selectable for new bookings |

Product Management UI

- /admin/products — list all products, filter by type / floor / status
- /admin/products/create — creation form
- /admin/products/{id}/edit — edit form
- Multi-photo upload: drag-to-reorder, upload to S3 via signed URL, photos stored as ordered JSON array
- Toggle active/inactive — does not delete. Inactive products hidden from public website.
- Accessible by Super Admin and Admin only

7.2 Conference Hall Booking

7.2.1 Booking Status Flow

Conference hall bookings pass through a mandatory 4-step approval chain before being confirmed.

| Status | Meaning | Trigger | | --- | --- | --- | | draft | Initial state when booking is submitted (online form or by staff) | — (not yet in approval queue) | | admin_pending | Booking submitted to Admin for first review | Admin or Operations creates booking → Admin notified | | accountant_pending | Admin has approved; awaiting Finance final approval | Admin clicks Approve → Finance notified | | booking_approved | Finance has given final approval — booking is confirmed | Finance clicks Approve → Client notified (WhatsApp + email) | | rejected | Rejected at any approval stage | Admin or Finance clicks Reject → rejection reason required → Client notified | | waitlisted | Slot is unavailable; client added to waiting list | System sets status when conflict detected | | cancelled | Approved booking subsequently cancelled | Admin or Finance records cancellation → Client notified | | rescheduled | Booking dates changed after approval | Status resets to draft; new approval chain required |

On every status transition: (1) record in booking_status_logs table, (2) dispatch queue job to send WhatsApp + email notification to client.

7.2.2 Booking Record — bookings Table

| Field | Type | Notes | | --- | --- | --- | | id | bigint | PK | | booking_code | varchar unique | System-generated: BK-2026-0042 | | type | enum | conference_hall / office_lease / educational_lease | | product_id | bigint FK | The space being booked | | client_name | varchar | Full name of the organiser/client | | client_company | varchar nullable | Company or organisation name | | client_email | varchar | For notifications and confirmation | | client_phone | varchar | For WhatsApp notifications | | client_national_id | varchar nullable | For KYC compliance | | session_type | enum | morning / afternoon / evening / custom | | booking_date | date | Date of the event (single date or start date for multi-day) | | start_time | time | Effective start time | | end_time | time | Effective end time | | recurring | boolean | Whether this is a recurring booking series | | recurrence_rule | jsonb nullable | {frequency, days[], end_date} | | recurrence_group_id | bigint nullable | Links all occurrences in a series | | catering_package_id | bigint FK nullable | Selected catering package | | dj_requested | boolean | Default false | | cameraman_requested | boolean | Default false | | extra_services | jsonb nullable | Array of additional service objects | | base_price | decimal(15,2) | Space rental price | | catering_price | decimal(15,2) | Catering package price | | dj_price | decimal(15,2) | DJ add-on price (0 if not requested) | | cameraman_price | decimal(15,2) | Cameraman add-on price | | extras_price | decimal(15,2) | Sum of any other extras | | total_price | decimal(15,2) | Sum of all price components | | status | enum | draft / admin_pending / accountant_pending / booking_approved / rejected / waitlisted / cancelled / rescheduled | | payment_status | enum | unpaid / partial / paid | | notes | text nullable | Internal notes | | rejection_reason | text nullable | Required when status = rejected | | created_by_user_id | bigint FK nullable | Null if submitted via public form | | created_at / updated_at | timestamps | | deleted_at | timestamp nullable | Soft delete |

7.2.3 Admin Booking Management

- /admin/bookings — master list with tab navigation: All / Draft / Admin Pending / Accountant Pending / Approved / Rejected / Cancelled
- Pending Approvals widget for Admin: shows all admin_pending bookings; each has Approve and Reject action buttons inline
- Pending Approvals widget for Finance: shows all accountant_pending bookings; each has Approve and Reject action buttons inline
- Admin cannot see or act on accountant_pending queue — that belongs to Finance
- Booking calendar: react-big-calendar or FullCalendar component. Views: month, week, day. Color codes: draft=grey, pending=amber, approved=green, rejected=red, cancelled=grey-strikethrough
- Create booking on behalf of client: available to Operations and Admin. Same form as public form but with additional internal fields (notes, priority flag)
- Booking detail page: full booking info, status history timeline (from booking_status_logs), communication log, all linked invoices with payment status
- Availability check: before creating or approving, system validates no overlapping approved bookings for the same product_id + date + session_type

7.2.4 Waiting List

- Triggered when: a booking request is submitted for a product/session/date that already has an approved booking
- waiting_list table: id, product_id, session_type, booking_date, client_name, client_email, client_phone, created_at
- Position is queue-based (FIFO by created_at)
- On booking cancellation: system queries waiting_list WHERE product_id = X AND session_type = Y AND booking_date = Z ORDER BY created_at ASC LIMIT 1
- Sends WhatsApp + email to first in queue: 'A slot has opened for [Hall Name] on [Date] — [Session]. Reply or click to confirm your interest within 24 hours.'
- If first contact does not respond within 24 hours, notification moves to next in queue (cron job)

7.2.5 Recurring Bookings

- Staff can mark a booking as recurring at creation time

- recurrence_rule JSON: { 'frequency': 'weekly' | 'monthly', 'days': ['monday', 'wednesday'], 'end_date': '2026-12-31' }
- On save: Laravel job CreateRecurringBookings dispatched immediately to queue
- Job creates one bookings record per occurrence, all linked by recurrence_group_id
- Each occurrence starts in draft status and goes through its own approval chain
- Cancelling one occurrence: only that record's status set to cancelled
- Cancelling entire series: UI prompt 'Cancel this occurrence or all future occurrences?' — if all, bulk update WHERE recurrence_group_id = X AND booking_date >= today
- Rescheduling a single occurrence: updates that booking's date/time, resets status to draft for new approval

7.3 Office & Educational Space Leasing

7.3.1 Lease Booking Flow

Office and educational leases use a simplified approval flow — Admin approval only (Finance is not a required approver, as Finance handles the billing separately).

- draft → admin_pending: when Operations creates a lease booking
- admin_pending → lease_active: Admin approves. System creates the lease record and triggers tenant onboarding.
- admin_pending → rejected: Admin rejects. Rejection reason required.
- lease_active → expired: system auto-sets when today > lease.end_date (daily cron job)
- lease_active → terminated: Admin terminates lease early. Records termination date and reason.

7.3.2 Tenant Onboarding

- Triggered when Admin approves a lease booking
- Admin completes tenant profile: company_name, contact_person_name, email, phone, national_id
- Documents uploaded to S3, referenced in tenant_documents table: lease agreement, KYC docs, business registration, notarised documents
- Portal access: Admin can generate portal credentials (email + auto-generated password) for the tenant. Password sent via email (Resend) and WhatsApp.
- tenants table type field: office / educational / conference_client
- Tenant status workflow: pending → active (on lease approval) → terminated

7.3.3 Lease Record — leases Table

Field	Type	Notes
id	bigint	PK
tenant_id	bigint FK	The tenant
space_id	bigint FK	The space (product)
lease_code	varchar unique	e.g. LS-2026-0007
start_date	date	Lease commencement date
end_date	date	Lease expiry date
monthly_rent	decimal(15,2)	For monthly leases
semester_amount	decimal(15,2) nullable	For educational semester leases
billing_cycle	enum	monthly / semester
status	enum	active / expired / terminated
security_deposit_amount	decimal(15,2)	Amount collected upfront
security_deposit_status	enum	held / returned / applied
contract_signed_online	boolean	Whether e-signed in system
contract_file_url	varchar nullable	S3 URL of signed contract PDF
external_contract_url	varchar nullable	S3 URL of externally signed contract
created_by_user_id	bigint FK	Who created the lease
created_at / updated_at	timestamps	
deleted_at	timestamp nullable	Soft delete

7.3.4 Automated Lease Reminders

A Laravel scheduled command LeaseExpiryReminder runs daily via cron. It queries leases WHERE status = active AND end_date <= today + 10 days AND renewal_reminder_sent = false. For each match: dispatches queue job SendLeaseRenewalReminder which sends WhatsApp + email to tenant and email to Admin. Sets renewal_reminder_sent = true on the lease record to prevent duplicate sends.

SECTION 8 — PHASE 3: FINANCE & ACCOUNTING

This is the core financial engine. It follows double-entry accounting principles — every financial event results in a journal entry with balanced debit and credit lines. Finance Officer has exclusive write access; Admin and Super Admin have read-only access to financial data.

8.1 Chart of Accounts

Defines all accounts used in the double-entry ledger. Pre-seeded with the default structure below. Finance can add sub-accounts; Super Admin can edit account codes.

Code	Account Name	Type	Notes	---	---	---	---	1001	Cash (Petty Cash)	Asset	—	1010	Edahab - Conference Halls	
Asset	Mobile money for hall bookings	1011	ZAAD - Conference Halls	Asset	Mobile money for hall bookings	1012	Edahab - Office Rentals	Asset	Mobile money for office rent	1013	ZAAD - Office Rentals	Asset	Mobile money for office rent	
1020	Darasalam Bank (Main)	Asset	Primary bank account	1100	Accounts Receivable - Office Tenants	Asset	Outstanding rent	1101	Accounts Receivable - Conference Clients	Asset	Outstanding hall invoices	1200	Security Deposits Held	
Asset	Deposits collected from tenants	1300	Prepaid Expenses	Asset	Advance payments made	2001	Accounts Payable - Vendors	Liability	Outstanding vendor bills	2100	Security Deposits Owed	Liability	Deposits to be returned to tenants	
2200	Salary Payable	Liability	Accrued salaries not yet disbursed	3001	Conference Hall Rental Income	Revenue	3002	Office Rental Income	Revenue	3003	Educational Facility Income	Revenue	3010	Catering Revenue (Silver)
Revenue	3011	Catering Revenue (Gold)	Revenue	3012	Catering Revenue (Platinum)	Revenue	3020	DJ / Cameraman Revenue	Revenue	3030	Additional Services Revenue	Revenue	Cleaning, internet, etc.	
4001	Electricity (Utility)	Expense	4002	Water (Utility)	Expense	4003	Sewage (Utility)	Expense	4004	Garbage Collection	Expense	4010	Salaries - Internal Staff	
Expense	4011	Salaries - Maintenance	Expense	4012	Salaries - Cafeteria / Restaurant	Expense	4020	Maintenance & Repairs	Expense	4030	Catering Supplies / Costs	Expense	4031	DJ / Cameraman Costs
Expense	4040	Cleaning Supplies	Expense	4050	Office Supplies	Expense	4060	Internet & Telecommunications	Expense	4070	Vendor - Event Materials	Expense	4080	Miscellaneous Expenses
Expense														

8.2 Account Management (Bank & Mobile Money)

The 5 operating accounts are pre-created as both chart_of_accounts entries AND tracked accounts records. Current balance is always computed from transaction history — never stored as a static field.

accounts Table

Field	Type	Notes	---	---	---	id	bigint	PK	chart_of_account_id	bigint FK	Links to COA entry	name	varchar	Display name (e.g. 'Edahab - Conference Halls')
type	enum	bank / mobile_money / cash	account_identifier	varchar	Account number or mobile money number	active	boolean	Whether account is available for transactions	notes	text nullable	e.g. 'Used exclusively for hall booking payments'			

account_transactions Table

Field	Type	Notes	---	---	---	id	bigint	PK	account_id	bigint FK	Which account	type	enum	credit / debit
amount	decimal(15,2)	Positive value always	description	varchar	Human-readable description	reference_code	varchar	INV/VB/EXP/TRF code etc.	journal_entry_id	bigint FK nullable	Linked journal entry	transaction_date	date	Date of transaction
created_by	bigint FK	User who recorded it	created_at	timestamp										

Inter-Account Transfers

- Finance UI at /admin/accounts/transfer: select source account, destination account, amount, date, notes
- System creates: account_transactions DEBIT on source, account_transactions CREDIT on destination
- System posts journal entry: Debit destination COA code, Credit source COA code

- Transfer reference code: TRF-2026-0003 (generated by ReferenceCodeService)
- Cannot transfer to/from inactive accounts
- Confirmation modal required before processing

8.3 Bill Code & Reference System

Every financial document receives a unique auto-generated reference code. Codes are sequential per year per prefix — no gaps, no duplicates.

| Document Type | Prefix | Example Code | | --- | --- | --- | | Customer Invoice | INV | INV-2026-0042 | | Vendor Bill | VB | VB-2026-0009 | | Purchase Order | PO | PO-2026-0005 | | Expense Record | EXP | EXP-2026-0017 | | Electricity Bill | ELEC | ELEC-2026-0011 | | Payment Receipt | RCP | RCP-2026-0088 | | Payslip | PAY | PAY-2026-0034 | | Lease | LS | LS-2026-0007 | | Booking | BK | BK-2026-0042 | | Inter-Account Transfer | TRF | TRF-2026-0003 | | Journal Entry | JE | JE-2026-0156 | | Payroll Run | PR | PR-2026-0004 | | Maintenance Request | MR | MR-2026-0019 |

Implementation: reference_sequences table (id, prefix, year, last_sequence). ReferenceCodeService::generate(\$prefix) — opens DB transaction, SELECT ... FOR UPDATE on the sequence row, increments last_sequence, formats as PREFIX-YEAR-XXXX, commits. This prevents duplicate codes under concurrent requests.

8.4 Customer Invoices & Billing

invoices Table

| Field | Type | Notes | | --- | --- | --- | | id | bigint | PK | | invoice_code | varchar unique | INV-2026-0042 | | type | enum | office_rent / educational / conference_hall / electricity / manual | | tenant_id | bigint FK | The billed tenant | | lease_id | bigint FK nullable | For rent invoices | | booking_id | bigint FK nullable | For conference hall invoices | | issue_date | date | Date invoice was created | | due_date | date | Calculated from issue_date + invoice_due_days setting | | billing_period_start | date nullable | For rent: first day of billing month | | billing_period_end | date nullable | For rent: last day of billing month | | lpo_number | varchar nullable | Local Purchase Order number for corporate/govt clients | | subtotal | decimal(15,2) | Before any tax (tax defaulted to 0 in Pilot) | | total_amount | decimal(15,2) | Final billed amount | | status | enum | draft / sent / paid / partial / overdue | | payment_account_id | bigint FK nullable | Preferred receiving account | | notes | text nullable | | sent_at | timestamp nullable | When first emailed/WhatsApp'd to tenant | | created_by | bigint FK | User who created it | | created_at / updated_at | timestamps | | deleted_at | timestamp nullable | Soft delete |

Invoice Generation Methods

- Auto — Monthly Rent: Laravel scheduled command AutoGenerateMonthlyInvoices runs on the 1st of every month at 07:00. Queries all leases WHERE status = active AND billing_cycle = monthly. Creates one invoice per active lease. Dispatches to queue to prevent HTTP timeout.
- Auto — Educational: same command checks billing_cycle = semester, creates invoice on the first day of each semester period based on lease start_date.
- Auto — Conference Hall: triggered when a booking transitions to booking_approved. InvoiceService::createForBooking(\$booking) is called by the BookingApprovalService.
- Manual — Electricity: Finance navigates to /admin/invoices/create, selects type = electricity, links an electricity_reading record as a line item.
- Manual — Ad-hoc: Finance creates a one-off invoice for any purpose via /admin/invoices/create.

Invoice PDF (DomPDF)

- Blade template: resources/views/pdfs/invoice.blade.php
- Design: Haleelo Tower logo top-left, navy header bar, gold accents, Poppins font embedded
- Contents: invoice_code, issue_date, due_date, LPO number (if set), tenant name and address, line items table (description / qty / unit price / line total), subtotal, total amount, payment instructions listing all 5 account options with

account identifiers

- Generated on demand when Finance clicks 'Download PDF' or when invoice is sent
- Stored in S3 at invoices/{invoice_code}.pdf after first generation
- Tenant can download from portal (signed S3 URL, expires in 1 hour)

Invoice Delivery

- Email: Resend API sends HTML email with PDF attached. Template: resources/views/emails/invoice.blade.php
- WhatsApp: message body includes booking/invoice summary + portal link to download PDF
- Delivery tracked: invoices.sent_at updated on first send
- Finance can resend at any time from the invoice detail page

Automated Invoice Reminders

- 3 days before due date (if status != paid): dispatch SendInvoiceReminderJob → WhatsApp + email
- On due date (if status != paid): dispatch → WhatsApp + email
- 1 day after due date (if status != paid): status auto-updated to overdue, dispatch → WhatsApp + email + add to overdue dashboard widget
- 7 days overdue: dispatch escalation email to Admin
- All reminders sent via queue (Redis + Horizon) — never block the main request cycle

8.5 Vendor Bills & Purchase Orders

Tracks all procurement: purchase orders sent to vendors, and bills received when goods/services are delivered.

- Vendors table: tracks supplier master data — name, contact, category (food_supplier / cleaning / equipment / event_materials / other), active status
- Purchase Orders: created before ordering goods. Status flow: draft → sent → received → billed → cancelled
- Vendor Bills: created when physical invoice/bill is received from vendor. Can be linked to a PO or standalone.
- Bill payment: Finance records payment → system creates account_transaction and posts journal entry (Debit AP, Credit payment account)
- Receipt scanning: vendor bill's receipt_file_url accepts a photo upload to S3 (for physical receipt archiving)
- purchase_order_items and vendor_bill_items tables store line-item breakdown with expense_account_id FK to COA

8.6 Expense Recording

Direct expenses (not vendor bills) — e.g. petty cash purchases, utility payments, one-off costs.

- Finance records expenses at /admin/expenses/create
- Fields: description, amount, expense_date, expense_account_id (COA FK), payment_account_id (which account was used to pay), booking_id (optional — links event expenses to a booking for profitability tracking), receipt photo upload (S3)
- On save: system auto-posts journal entry — Debit expense_account_id, Credit payment_account_id
- Expense code generated: EXP-2026-0017
- Expenses linked to a booking contribute to the per-event profit/loss calculation on the booking detail page

8.7 Electricity Bill Management

Each office tenant is metered individually. Finance records readings monthly and generates charges either as standalone electricity invoices or as line items on the monthly rent invoice.

- Step 1: Finance navigates to /admin/electricity/new-reading

- Step 2: Selects tenant and space from dropdowns
- Step 3: System auto-fills previous_reading from the last recorded reading for that tenant+space
- Step 4: Finance enters current_reading (kWh) and reading_date
- Step 5: System calculates: kwh_consumed = current - previous. Looks up rate active on reading_date from electricity_rates.
- Step 6: System calculates: total_charge = kwh_consumed × rate_per_kwh (snapshot)
- Step 7: Finance saves reading — status = 'recorded'
- Step 8: Finance can then: (a) add to next monthly invoice as a line item, OR (b) generate a standalone ELEC invoice
- Step 9: When added to invoice: invoice_line_items record created with electricity reading reference; electricity_readings.invoice_id set; status updated to 'invoiced'
- Step 10: electricity_readings.rate_per_kwh is a snapshot — rate changes do not retroactively alter recorded readings

8.8 Payment Recording

All customer receipts and vendor payments are recorded here. Each payment record triggers account balance updates and journal entry posting.

| Field | Type | Notes | | --- | --- | --- | | id | bigint | PK | | payment_code | varchar unique | RCP-2026-0088 | | type | enum | customer_receipt / vendor_payment | | invoice_id | bigint FK nullable | For customer receipts | | vendor_bill_id | bigint FK nullable | For vendor payments | | amount | decimal(15,2) | Amount received/paid | | payment_date | date | Actual date of transaction | | payment_method | enum | edahab / zaad / bank_transfer / cheque / cash | | account_id | bigint FK | Which account received or paid | | reference_number | varchar nullable | Mobile money or bank transaction reference | | notes | text nullable | | created_by | bigint FK | | created_at | timestamp | |

Payment Processing Logic

- Finance records payment via /admin/payments/create or from the invoice detail page
- System checks: amount vs invoice.total_amount — already_paid. Sets invoice status = paid (if full) or partial (if less than total)
- Journal entry auto-posted by PaymentService → AccountingService:
- Customer receipt: Debit account_id's COA code (e.g. 1010 Edahab-Halls), Credit 1100 or 1101 Accounts Receivable
- Vendor payment: Debit 2001 Accounts Payable, Credit account_id's COA code
- account_transactions record created simultaneously
- Security deposit return: Finance records via /admin/deposits/{id}/return. Posts: Debit 2100 Security Deposits Owed, Credit payment account COA code

8.9 General Ledger & Journal Entries

Every financial event automatically creates a journal entry. Finance can also create manual journal entries for adjustments.

Auto-Generated Journal Entry Triggers

- Invoice created (sent status): Debit Accounts Receivable, Credit Revenue account(s)
- Payment received: Debit Cash/Bank account, Credit Accounts Receivable
- Vendor bill created: Debit Expense account, Credit Accounts Payable
- Vendor payment made: Debit Accounts Payable, Credit Cash/Bank account
- Expense recorded: Debit Expense account, Credit Cash/Bank account
- Security deposit received: Debit Cash/Bank account, Credit Security Deposits Owed
- Security deposit returned: Debit Security Deposits Owed, Credit Cash/Bank account
- Inter-account transfer: Debit destination account, Credit source account
- Payroll finalized: Debit Salary Expense accounts, Credit Salary Payable or Cash/Bank

Journal Views

- /admin/accounting/journal — all entries chronological. Columns: date, JE code, description, debit account, credit account, amount, source (auto/manual), posted by. Filterable by date range and source.
- /admin/accounting/ledger — per-account view. Filter by account and date range. Shows: date, JE code, description, debit amount, credit amount, running balance. Exportable.
- Trial Balance (/admin/accounting/trial-balance): all accounts with total debits, total credits, balance. Sum of all debits must equal sum of all credits — displayed as a validation check.

8.10 HR & Payroll

Full payroll module for three employee departments: Internal Staff, Maintenance, Cafeteria/Restaurant. Finance has exclusive access.

Payroll Calculation Formula

For salaried employees: $\text{Net Pay} = (\text{base_salary} \times (\text{days_worked} / \text{working_days_in_month})) + \text{overtime_pay} - \text{total_deductions}$
For daily rate employees: $\text{Net Pay} = (\text{daily_rate} \times \text{days_worked}) + \text{overtime_pay} - \text{total_deductions}$

Payroll Workflow

- Step 1: Finance opens /admin/payroll/new-run, selects month (YYYY-MM) and optionally filters by department
- Step 2: System loads all active employees with their attendance_logs for the selected month
- Step 3: Finance reviews attendance summary — can manually adjust days_worked, add overtime records, add deductions
- Step 4: Finance clicks 'Calculate' — system computes net pay per employee using the formula above
- Step 5: Finance reviews payslip previews on screen
- Step 6: Finance clicks 'Finalize & Send' — system: generates PDF payslip per employee (DomPDF, stored in S3), dispatches queue jobs to deliver via WhatsApp + email, posts payroll journal entry
- Step 7: payroll_runs.status changes from draft to finalized
- Step 8: Finalized payroll runs cannot be deleted — only voided with a reversal journal entry

Payslip PDF Contents

- Employee name, employee code, job title, department
- Month and payroll run code
- Attendance summary: working days in month, days worked, days absent
- Earnings breakdown: base salary (prorated), overtime pay, gross pay
- Deductions breakdown: unpaid absence, salary advance, disciplinary, total deductions
- Net pay (highlighted in navy/gold branded box)
- Haleelo Tower branding: logo, address, Poppins font
- Stored in S3: payslips/{PAY-code}.pdf

8.11 Financial Reporting

All reports are filterable by date range and exportable as PDF (DomPDF) and Excel (Maatwebsite/Laravel-Excel). Finance has full read/export access; Admin and Super Admin have read-only access.

Statement Reports

- Balance Sheet: assets, liabilities, equity at a selected date. Uses COA hierarchy to group sub-accounts. Shows current period vs prior period comparative.

- Profit & Loss: revenue vs expenses for a selected period. Groups by COA type. Net profit/loss calculated and highlighted.
- Cash Flow Statement: operating, investing, and financing activities. Derived from account_transactions and journal_entry_lines.

Audit / Ledger Reports

- Trial Balance: all COA accounts, total debits, total credits, balance. Validation indicator: green checkmark if Σ debits = Σ credits.
- General Ledger: per-account drill-down with all journal entry lines and running balance.
- Journal Audit: complete chronological list of all journal entries with debit/credit details.

Partner Reports

- Partner Ledger: per-tenant or per-vendor — all invoices, payments, and outstanding balance.
- Aged Receivables: outstanding customer invoices grouped by age: 0-30 days / 31-60 / 61-90 / 90+ days overdue.
- Aged Payables: outstanding vendor bills by the same age buckets.

Operational Reports

- Revenue Report: total revenue by source (conference halls, offices, educational, catering, add-ons). Monthly and annual breakdowns.
- Expense Report: total expenses by COA category. Compare to prior period.
- Occupancy Report: per floor, per space, per period — how many days/sessions each space was booked vs available.
- Bookings Report: filterable list of all bookings with status, revenue, client info. Export to Excel.
- Payment Report: all payments with status (Paid / Partial / Overdue), per account breakdown.
- Electricity Billing Report: per tenant, per period — readings, kWh consumed, charges, invoice status.
- Payroll Summary: per department, per month — gross pay, deductions, net pay totals.
- Invoice Analysis: by type, status, and period.

Management Dashboard KPIs

- Total Revenue — current month (sum of paid invoices)
- Active Tenants count
- Pending Bookings — count per status (admin_pending, accountant_pending)
- Overdue Invoices — count and total USD amount
- Account Balances — all 5 accounts displayed as cards (computed from account_transactions)
- Revenue Trend — bar chart showing last 12 months revenue
- Revenue by Source — doughnut chart (halls vs offices vs educational vs add-ons)
- Demand Analytics — top 3 most-booked spaces, most popular session type
- All KPIs computed on page load (no stale cache) — Redis cache with 5-minute TTL acceptable

Automated Monthly Report

Laravel scheduled command GenerateMonthlyReport runs on the last day of each month at 23:30. Compiles: total revenue, total expenses, net profit/loss, booking counts per status, overdue invoice total, top-performing space. Sends HTML email summary via Resend to Admin and Super Admin email addresses. Report also saved as a PDF in S3.

SECTION 9 — PHASE 4: MARKETING & PUBLIC WEBSITE

Built as a Next.js 14 App Router application. Static generation for most pages. ISR (Incremental Static Regeneration) for availability data with a short revalidation window (60 seconds). No authentication required. Designed mobile-first with Tailwind CSS using the brand palette.

9.1 Pages & Routes

| Route | Page Name | Key Features | | --- | --- | --- | | / | Homepage | Hero section (full-width image/video, headline, CTA buttons). Building intro paragraph. Space showcase cards (3 categories: offices, halls, educational). How-It-Works section (4 steps including the Finance approval step). Testimonials/Trust section. CTA banner. | | /spaces | Browse Spaces | Filter panel: type (hall/office), floor, capacity, session time, amenities. Space cards grid. Visual floor plan (SVG-based, color-coded: green=available, red=fully booked, grey=occupied by long-term tenant). Clicking a space card links to /spaces/[slug]. | | /spaces/[slug] | Space Detail Page | Photo gallery (lightbox). Full description. Floor, capacity, amenities chips. Session times with pricing per session. Catering packages (Silver/Gold/Platinum) with included items and price. DJ and Cameraman add-on pricing. 'Request Booking' CTA. | | /book/[slug] | Booking Request Form | Step 1: select session type (Morning/Afternoon/Evening/Custom). Step 2: select date(s). Step 3: add-ons (catering package dropdown, DJ checkbox + price, Cameraman checkbox + price). Step 4: client details (name, company, email, phone, national ID optional). T&C checkbox. Submit button. Live price summary sidebar updating as selections change. | | /book/confirmation | Booking Confirmation | Success screen: booking reference code, message 'Your request has been received. You will be contacted via email and WhatsApp within 24 hours.' Timeline showing the 4-step approval process with current status highlighted. | | /waiting-list/[slug] | Waiting List Form | Simple form: client name, email, phone, preferred session type, preferred date. Submit adds to waiting_list table. | | /faq | FAQ Page | Accordion-style Q&A covering: booking process, approval timeline, payment methods, catering options, cancellation policy, accessibility. | | /contact | Contact Page | Building address, phone, email, WhatsApp link, Google Maps embed (or static map image), office hours. |

9.2 Design System (Public Website)

- Tailwind CSS configured in tailwind.config.js with brand colors: primary: '#1B2D4F', accent: '#C9A052', darkNavy: '#0f1d33'
- Font: Poppins loaded via next/font/google — weights 400, 500, 600, 700
- Fully responsive: mobile-first breakpoints (sm: 640, md: 768, lg: 1024, xl: 1280)
- Navigation: sticky top nav with logo left, links right, 'Book Now' gold CTA button
- Footer: dark navy background, building info, links, social icons
- All images and videos stored in S3, served via CDN URL

9.3 Booking Form API Flow

- Step 1: User completes form and clicks Submit
- Step 2: Next.js frontend: client-side validation (required fields, valid email, future date), then POST to /api/bookings on Laravel API
- Step 3: Laravel BookingController::store(): validate request, call BookingService::createFromPublicForm(\$data)
- Step 4: BookingService: create bookings record (status = draft), generate BK code via ReferenceCodeService
- Step 5: Dispatch queue job SendBookingAcknowledgementJob: sends Resend email + WhatsApp to client with booking reference code
- Step 6: API returns { booking_code: 'BK-2026-XXXX', status: 'draft' }
- Step 7: Frontend redirects to /book/confirmation?code=BK-2026-XXXX
- Step 8: Admin is notified in the dashboard and receives an email about the new booking in the queue

SECTION 10 — PHASE 5: OTHER MODULES

10.1 Tenant Portal (portal.halelotower.so)

A separate Next.js 14 app. Authenticated with email + password (no 2FA). Tenants can only see their own data — enforced at the API level by scoping all queries to the authenticated tenant's ID.

| Page | Description | | --- | --- | | Dashboard | Lease summary card (space name, floor, lease period, next payment due date). Notification bell with unread count. Quick links to Invoices, Documents, Maintenance. | | Invoices | List of all invoices for this tenant. Columns: invoice code, type, billing period, amount, due date, status badge (colour-coded). PDF download button per invoice (signed S3 URL). Filter by status and date range. | | Electricity Bills | Separate list showing electricity_readings records. Columns: period, previous reading, current reading, kWh consumed, rate, charge, linked invoice status. | | Bookings | For conference_client type tenants: list of their hall bookings with status badges. Shows the 4-step approval progress bar. | | Documents | Downloadable signed lease agreement and any other uploaded documents. Each file shows document type, upload date, download button (signed S3 URL). | | Maintenance | Submit new request form: category (AC/electrical/cleaning/plumbing/equipment/other), description textarea, photo upload. List of all submitted requests with status badges (open/in_progress/resolved) and any admin notes. | | Profile | Update contact phone number. Change password form. | | Announcements | Read building announcements. Unread announcements highlighted. |

Tenant Portal Notifications

- tenant_notifications table: id, tenant_id, message, type (invoice / booking / maintenance / announcement), read (boolean), link (URL), created_at
- Bell icon in portal nav shows unread count
- Polling: frontend polls GET /api/tenant/notifications?unread=true every 30 seconds
- Clicking a notification: marks as read (PATCH /api/tenant/notifications/{id}/read), navigates to linked page
- Triggered by: new invoice created, booking status change, maintenance request status change, new announcement broadcast

10.2 Maintenance Management

maintenance_requests Table

| Field | Type | Notes | | --- | --- | --- | | id | bigint | PK | | request_code | varchar unique | MR-2026-0019 | | tenant_id | bigint FK | Requesting tenant | | space_id | bigint FK | Affected space | | category | enum | ac / electrical / cleaning / plumbing / equipment / other | | description | text | Detailed description of the issue | | photo_url | varchar nullable | S3 URL of uploaded photo | | status | enum | open / in_progress / resolved | | assigned_to_user_id | bigint FK nullable | Internal staff assigned to handle it | | admin_notes | text nullable | Internal notes visible to staff only | | resolved_at | timestamp nullable | When status changed to resolved | | created_at / updated_at | timestamps | |

Maintenance Workflow

- Tenant submits via portal → status = open → Admin and Finance receive email + in-app notification
- Admin opens /admin/maintenance, assigns staff member (assigned_to_user_id), status → in_progress → tenant notified (WhatsApp + email)
- Assigned staff marks as resolved → status → resolved → tenant notified (WhatsApp + email)
- All status changes recorded in audit_logs
- /admin/maintenance list: tabs Open / In Progress / Resolved. Filter by category, tenant, space.

10.3 Communications & Announcements

Broadcast Announcements

- Finance or Admin creates announcement at /admin/communications/new
- Fields: subject, body (rich text), recipient group (all_tenants / office_tenants / educational_tenants / conference_clients), channels (email, WhatsApp, or both)

- On submit: system queries tenant IDs matching the recipient group, dispatches SendAnnouncementJob to queue
- Queue job loops through recipients: sends Resend email + WhatsApp per tenant, creates tenant_notifications record per tenant
- announcements table records the broadcast with sent_at timestamp
- Delivery log: admin can view which tenants received the announcement (join on tenant_notifications)

All Automated Notification Triggers

| Trigger | When | Channel | Recipient | | --- | --- | --- | --- | | Lease renewal approaching | 10 days before lease.end_date | WhatsApp + Email | Tenant + Admin | | Rent invoice created | When invoice created and sent | WhatsApp + Email | Tenant | | Invoice reminder | 3 days before due date (if unpaid) | WhatsApp + Email | Tenant | | Invoice due today | On due date (if unpaid) | WhatsApp + Email | Tenant | | Invoice overdue | 1 day past due date | WhatsApp + Email | Tenant + Admin | | Invoice 7+ days overdue | 7 days past due date | Email | Admin | | Booking acknowledgement | On booking submission | WhatsApp + Email | Client | | Booking status change | On every status transition | WhatsApp + Email | Booking client | | Maintenance: in progress | When request assigned | WhatsApp + Email | Tenant | | Maintenance: resolved | When request resolved | WhatsApp + Email | Tenant | | Waiting list slot opens | When approved booking cancelled | WhatsApp + Email | Next in queue | | Payslip delivery | On payroll finalization | WhatsApp + Email | Employee | | Monthly report | Last day of month | Email | Admin + Super Admin | | New booking in queue | When booking reaches admin_pending | Email | Admin | | Booking Finance queue | When booking reaches accountant_pending | Email | Finance |

SECTION 11 — TECHNOLOGY STACK

| Layer | Technology | Version | Notes | | --- | --- | --- | --- | | Frontend Framework | Next.js (App Router) | 14.x | TypeScript, Server Components, API routes. Used for all 3 frontend apps. | | UI Styling | Tailwind CSS | 3.x | Brand colors in tailwind.config.js. Mobile-first. | | Font | Poppins | via next/font/google | Weights: 400, 500, 600, 700. Fallback: Calibri in docx, system-sans in web. | | Backend Framework | Laravel | 11.x | PHP 8.3. REST API. All business logic in app/Services/. | | Authentication | Laravel Sanctum | — | SPA cookie tokens. HTTP-only, Secure, SameSite=Lax. | | Permissions | Spatie Laravel Permission | — | Roles and permissions. Middleware on all API routes. | | Database | PostgreSQL | 16 | Primary datastore. decimal(15,2) for all money. Soft deletes. | | Cache / Queue Backend | Redis | 7 | Sessions, queues, rate limiting, KPI cache (5-min TTL). | | Queue Runner | Laravel Horizon | — | Monitors and processes queues. Supervised by Supervisor. | | PDF Generation | Laravel DomPDF (barryvdh) | — | Invoices, payslips, reports. Blade templates in resources/views/pdfs/. | | Excel Export | Maatwebsite/Laravel-Excel | 3.x | Report exports. Uses PhpSpreadsheet. | | File Storage | AWS S3 (or compatible) | — | Contracts, KYC docs, payslips, photos, receipts. Laravel Storage facade with signed URLs. | | Email | Resend API | — | resend/resend-php SDK. HTML Blade templates. Branded with logo and colors. | | WhatsApp | WhatsApp Business API | Twilio or 360dialog | Automated messages for all notification triggers. Provider switchable via config. | | Hosting | VPS (Ubuntu 22.04) | — | Nginx, PHP-FPM, PostgreSQL, Redis. PM2 for Next.js. Supervisor for Horizon. | | SSL | Let's Encrypt (Certbot) | — | All 3 subdomains + API domain. Auto-renewal. | | Version Control | Git | — | Recommended: separate repos per app or monorepo with workspaces. |

SECTION 12 — DEPLOYMENT & INFRASTRUCTURE

12.1 Domain Structure

halelotower.so — Public marketing website — Next.js app portal.halelotower.so — Tenant portal — Next.js app admin.halelotower.so — Admin dashboard — Next.js app api.halelotower.so — Laravel REST API — not publicly linked, CORS-restricted

12.2 Server Architecture

- Single VPS (Ubuntu 22.04). Can be split to multiple VPS for scale later.
- Nginx: reverse proxy. Each Next.js app runs on a distinct port (e.g. 3000, 3001, 3002). Nginx routes subdomain → port. Laravel served via PHP-FPM on a socket.

- PM2: process manager for Next.js apps. Auto-restarts on crash. Startup on server reboot.
- Supervisor: manages Laravel Horizon (queue worker). Auto-restarts. Config in /etc/supervisor/conf.d/horizon.conf
- Cron: one entry — '**** cd /var/www/api && php artisan schedule:run >> /dev/null 2>&1' — Laravel handles all scheduled command dispatch internally.
- PostgreSQL: running on localhost:5432. Daily pg_dump backup piped to S3 via cron at 02:00.
- Redis: running on localhost:6379. Used for sessions, queues, and cache.
- SSL: Certbot (Let's Encrypt). Certificates auto-renewed. Nginx configured with redirect HTTP → HTTPS.

12.3 Deployment Steps (VPS)

- Step 1: Clone repos to /var/www/public, /var/www/portal, /var/www/admin, /var/www/api
- Step 2: Install PHP 8.3, Composer, Node.js 20, NPM, Redis, PostgreSQL, Nginx, Certbot
- Step 3: Run composer install --no-dev for Laravel. Run npm ci && npm run build for each Next.js app.
- Step 4: Configure .env files for each app (never commit .env to git)
- Step 5: Run php artisan migrate --force && php artisan db:seed for initial database setup
- Step 6: Configure Nginx virtual hosts for all 4 domains
- Step 7: Obtain SSL certificates: certbot --nginx -d halelotower.so -d portal.halelotower.so -d admin.halelotower.so -d api.halelotower.so
- Step 8: Start PM2 for Next.js apps. Start Supervisor/Horizon. Set cron.
- Step 9: Verify all apps load and API is accessible from Next.js apps
- Step 10: Smoke test: create a test user, perform a test login with 2FA, create a test booking

12.4 Key Environment Variables

Laravel (.env)

APP_NAME="Haleelo Tower" APP_ENV=production APP_URL=https://api.halelotower.so APP_KEY=base64:...

DB_CONNECTION=pgsql DB_HOST=127.0.0.1 DB_PORT=5432 DB_DATABASE=haleelo_tower DB_USERNAME=haleelo_user
DB_PASSWORD=...

CACHE_DRIVER=redis QUEUE_CONNECTION=redis SESSION_DRIVER=redis REDIS_HOST=127.0.0.1 REDIS_PORT=6379

AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=... AWS_DEFAULT_REGION=us-east-1 AWS_BUCKET=haleelo-tower

RESEND_API_KEY=re... RESEND_FROM_ADDRESS=noreply@halelotower.so RESEND_FROM_NAME="Haleelo Tower"
RESEND_REPLY_TO=info@halelotower.so

WHATSAPP_PROVIDER=twilio # or 360dialog WHATSAPP_ACCOUNT_SID=... # Twilio WHATSAPP_AUTH_TOKEN=... # Twilio
WHATSAPP_SENDER_NUMBER=whatsapp:+252...

SANCTUM_STATEFUL_DOMAINS=halelotower.so,portal.halelotower.so,admin.halelotower.so SESSION_DOMAIN=.halelotower.so

Next.js apps (.env.local)

NEXT_PUBLIC_API_URL=https://api.halelotower.so NEXT_PUBLIC_APP_NAME="Haleelo Tower"

SECTION 13 — PHASED DELIVERY PLAN

Phase	Focus	Key Deliverables	Est. Duration
Phase 1	User Management & Settings	Auth system (email + password + 2FA), RBAC with Spatie, user CRUD, invite-by-email, central settings module (all 9 settings sections), audit trail	1 week
Phase 2	Booking System	Product management, catering package config, conference hall 4-step booking flow with approval chain, office/educational lease flow, booking calendar, waiting list, recurring bookings, availability validation	1.5 weeks
Phase 3	Finance & Accounting	Chart of accounts, 5-account management, bill code system, customer invoicing (auto + manual), vendor bills & POs, expense recording, electricity billing, payment recording, inter-account transfers, journal entries, trial balance, all 18 financial reports, HR & payroll full module	3 weeks
Phase 4	Public Website	All public pages (/spaces, /spaces/{slug}, /book/{slug}, /confirmation, /waiting-list, /faq, /contact), booking request form, floor map SVG, catering packages display, responsive Tailwind design	1 week
Phase 5	Tenant Portal, Maintenance, Communications	Full tenant portal (7 pages), maintenance module, broadcast announcements, all automated WhatsApp/email notification triggers, tenant notification bell, portal credential management	1 week
Final	Testing & Launch	End-to-end testing all roles and workflows, UAT with client, staff training sessions (per role), go-live deployment checklist, DNS cutover, SSL verification	0.5 weeks
TOTAL			~8 weeks

SECTION 14 — TRAINING & HANDOVER

Training will be conducted with each role separately. All training includes a recorded screen-share session and a written quick reference card per role.

Role	Who	Training Topics
Super Admin	IT/System Admin	Full system walkthrough — all settings, user management, audit trail, system recovery, .env management, Horizon dashboard, server basics
Admin	Building Manager	Booking calendar and approval workflow, tenant management, product management, reports (read-only), announcement broadcasts, lease renewals
Operations	Receptionist x2	Creating bookings on behalf of clients, tenant onboarding process, invoice creation, navigating the dashboard
Finance	Finance Officer	All finance workflows: invoice management, payment recording, vendor bills, expense recording, electricity billing, payroll run, all financial reports and exports
Tenants	Long-Term Tenants	Tenant portal: viewing invoices, downloading documents, submitting maintenance requests, reading announcements

Handover Package

- Source code repos (Git) with README for each app
- .env template files with all required keys documented
- Server configuration files (Nginx virtual hosts, Supervisor config)
- Database ERD (Entity Relationship Diagram) exported from PostgreSQL
- This Implementation Plan document (v3.0)
- Recorded training videos (one per role)
- Quick reference cards (PDF, one per role)
- Admin credentials for all 4 applications
- S3 bucket access credentials
- Resend API account login
- WhatsApp Business API credentials

SECTION 15 — POST-LAUNCH SUPPORT

A monthly retainer is recommended covering the following:

- Bug fixes: any defects discovered post-launch addressed within 48 hours (critical) or next scheduled update (minor)
- Server monitoring: uptime monitoring, SSL certificate renewal alerts, disk space alerts
- Platform updates: Laravel/Next.js security patches, dependency updates
- Feature additions: new requirements beyond this v3.0 spec scoped and quoted per request
- Backup verification: weekly confirmation that automated S3 backups are completing and restorable

- Queue health: monitoring Laravel Horizon dashboard for stuck jobs or failed queues
- Primary contact: Building Manager (Admin role) is the designated point of contact for all support requests
- Support channel: dedicated WhatsApp group or email thread per client preference

SECTION 16 — RECOMMENDATIONS FOR REVIEW

These features were identified as valuable additions not explicitly requested. Late payment penalties and bank reconciliation have been moved to Future Phase per client instruction. The remaining 12 recommendations are listed below for client review.

#	Recommendation	Phase
1	Petty Cash Management	Pilot
2	Receipt / Payment Vouchers	Pilot
3	Inter-Account Transfer Receipts	Pilot
4	Budget Planning vs Actuals	Future
5	Accounts Receivable Aging Alerts	Future
6	Fiscal Year Lock	Future
7	Catering Expense Tracking Per Event	Pilot
8	Asset Register	Future
9	Credit Notes	Future
10	Multi-Currency Display	Future
11	Recurring Expense Templates	Future
12	Tax / VAT Configuration	Future
13	Late Payment Penalties	Future
14	Bank Reconciliation Tool	Future

1. Petty Cash Management [Pilot]: Track small day-to-day cash purchases with a dedicated petty cash account (COA 1001), running balance display, and a formal replenishment workflow. Finance records petty cash disbursements and requests replenishment when the float falls below a configurable threshold.
2. Receipt / Payment Vouchers [Pilot]: Auto-generate a printable PDF voucher (DomPDF) for every payment received. Voucher includes: bill code (RCP-XXXX), amount, date, payment method, payer name, and a signature line. Sent to the client alongside the invoice as proof of payment.
3. Inter-Account Transfer Receipts [Pilot]: When funds are transferred between the 5 accounts (e.g. ZAAD → Darasalam Bank), auto-generate a transfer reference document (PDF) with the TRF code, source account, destination account, amount, date, and authorized by. Stored in S3.
4. Budget Planning vs Actuals [Future]: Allow Finance to set monthly or annual budget targets per expense COA category. Dashboard widget compares actual spend vs budget with a progress bar. Automated email alert when a category reaches 80% and again at 100% of budget.
5. Accounts Receivable Aging Alerts [Future]: Escalating automated alerts beyond the basic reminder: at 30 days, 60 days, and 90 days overdue — escalating from email-only to WhatsApp + email + Admin notification.
6. Fiscal Year Lock [Future]: Finance or Super Admin can lock a completed financial period so no retrospective edits can be made to historical journal entries. Locked periods display a padlock icon in the journal view.
7. Catering Expense Tracking Per Event [Pilot]: Already partially supported via `expenses.booking_id`. The booking detail page calculates: Event Revenue (invoice total) – Event Expenses (linked expenses) = Event Net Profit/Loss. This enables per-event profitability visibility.
8. Asset Register [Future]: Track the building's major assets (furniture, AV equipment, generator, AC units, etc.) with purchase date, cost, useful life, and residual value. System calculates and auto-posts monthly straight-line depreciation journal entries (Debit Depreciation Expense, Credit Accumulated Depreciation).
9. Credit Notes [Future]: Issue a formal credit note against an existing invoice for partial or full refunds. Credit note posts a reversing journal entry (Debit Revenue, Credit Accounts Receivable) and reduces the tenant's outstanding balance. Tenant can view credit notes in the portal.
10. Multi-Currency Display [Future]: Schema already designed to support a currency field (defaulting to USD). Activating multi-currency display (e.g. USD + Somali Shilling at a configurable exchange rate) would be a configuration switch requiring a minor frontend update.
11. Recurring Expense Templates [Future]: Define recurring expense templates that auto-generate each month (e.g. utility bills, monthly cleaning contracts). Saves Finance repetitive manual data entry. Templates configurable with: description, amount, expense account, payment account, day of month to generate.
12. Tax / VAT Configuration [Future]: Add a configurable `tax_rate` field (defaulting to 0%) to invoices. The system already calculates subtotal separately from `total_amount`. Activating tax would display a tax line on invoices and add a Tax Payable liability account to the COA.

13. Late Payment Penalties [Future]: FUTURE PHASE (per client instruction): configurable penalty amount or percentage applied automatically to invoices after a configurable grace period. Would generate a penalty invoice_line_item and post journal entries accordingly.
14. Bank Reconciliation Tool [Future]: FUTURE PHASE (per client instruction): a UI to upload or manually enter mobile money / bank statement transactions, match them against system account_transactions records, and flag discrepancies for Finance review.

SECTION 17 — COMPLETE FEATURE REGISTER

All confirmed platform features. Status: Pilot = included in this build. Future = scoped for a later phase.

#	Feature	Module	Phase	Status
1	Staff user account CRUD (create, edit, deactivate)	User Management	Pilot	
2	Role assignment: Super Admin, Admin, Operations, Finance	User Management	Pilot	
3	Spatie Laravel Permission package for RBAC	User Management	Pilot	
4	Invite user by email via Resend API with temporary password link	User Management	Pilot	
5	Staff login with email + password	Authentication	Pilot	
6	Two-factor authentication (2FA) via WhatsApp OTP for all staff	Authentication	Pilot	
7	2FA required on every login — no persistent bypass	Authentication	Pilot	
8	Laravel Sanctum SPA cookie token authentication	Authentication	Pilot	
9	Staff session timeout: 8 hours inactivity	Authentication	Pilot	
10	Tenant portal session timeout: 24 hours inactivity	Authentication	Pilot	
11	Password policy enforcement (uppercase, number, symbol, 8+ chars)	Authentication	Pilot	
12	Failed login logout: 5 attempts → 15 min lockout	Authentication	Pilot	
13	Password reset via Resend email link	Authentication	Pilot	
14	Audit trail: all state-changing actions logged to audit_logs	Audit	Pilot	
15	Audit log: immutable — no edit/delete by any role	Audit	Pilot	
16	Audit log: filterable by user, date, action, model type	Audit	Pilot	
17	Audit log: exportable as PDF and Excel	Audit	Pilot	
18	Central System Settings module (9 configuration sections)	Settings	Pilot	
19	Building name, logo, contact, address configuration	Settings	Pilot	
20	Timezone locked to EAT UTC+3	Settings	Pilot	
21	Email configuration: Resend API key, sender name/email	Settings	Pilot	
22	WhatsApp API configuration: Twilio or 360dialog	Settings	Pilot	
23	Session time configuration (Morning/Afternoon/Evening)	Settings	Pilot	
24	Payment terms: configurable invoice due days	Settings	Pilot	
25	Working hours per day setting (for payroll)	Settings	Pilot	
26	Fiscal year start month setting	Settings	Pilot	
27	Product management: create/edit conference halls, offices, educational space	Products	Pilot	
28	Product photos: multi-upload to S3, drag-to-reorder	Products	Pilot	
29	Product active/inactive toggle (no delete)	Products	Pilot	
30	Product amenities (JSON array: projector, AC, WiFi, etc.)	Products	Pilot	
31	Product services / add-ons per product	Products	Pilot	
32	Catering package config: Silver, Gold, Platinum (name, items, base price)	Products	Pilot	
33	DJ add-on pricing (system-wide setting)	Products	Pilot	
34	Cameraman add-on pricing (system-wide setting)	Products	Pilot	
35	Conference hall booking request (public form)	Bookings	Pilot	
36	Booking reference code generation (BK-YYYY-XXXX)	Bookings	Pilot	
37	4-step booking approval chain (Draft → Admin Pending → Accountant Pending → Approved)	Bookings	Pilot	
38	Admin approval action with rejection reason	Bookings	Pilot	
39	Finance final approval action with rejection reason	Bookings	Pilot	
40	Booking status log (full history of all status transitions)	Bookings	Pilot	
41	Booking calendar view (month/week/day, color-coded by status)	Bookings	Pilot	
42	Create booking on behalf of client (Operations / Admin)	Bookings	Pilot	
43	Booking detail page with status timeline and linked invoices	Bookings	Pilot	
44	Availability validation (no overlapping approved bookings)	Bookings	Pilot	
45	Waiting list for fully-booked slots	Bookings	Pilot	
46	Automatic waiting list notification on booking cancellation	Bookings	Pilot	
47	Recurring bookings with recurrence_rule (weekly/monthly)	Bookings	Pilot	
48	Cancel single occurrence vs cancel entire series	Bookings	Pilot	
49	Custom session duration booking (min 1 hour, max 1 month, up to 3 months advance)	Bookings	Pilot	
50	Office lease booking flow (draft → admin approval → lease active)	Bookings	Pilot	
51	Educational space lease booking flow (semester)	Bookings	Pilot	
52	Tenant onboarding: document collection (KYC, lease, business reg, notary)	Tenants	Pilot	
53	Tenant portal credential generation (email + password)	Tenants	Pilot	
54	Lease record with security deposit tracking	Tenants	Pilot	
55	Automated lease renewal reminder (10 days before expiry, WhatsApp + email)	Tenants	Pilot	
56	Lease expiry auto-detection (daily cron)	Tenants	Pilot	
57	Chart of accounts (COA) with default structure (1000-4999 range)	Finance	Pilot	
58	Finance can add sub-accounts; Super Admin can edit account codes	Finance	Pilot	
59	5 operating accounts (Edahabx2, ZAADx2, Darasalam Bank) with transaction tracking	Finance	Pilot	
60	Account balance computed from transaction history (no static field drift)	Finance	Pilot	
61	Per-account chronological transaction list with running balance	Finance	Pilot	
62	Inter-account	Finance	Pilot	

transfers with journal entry auto-posting | Finance | 3 | Pilot | | 63 | Bill code / reference code system (all 11 prefixes) | Finance | 3 | Pilot | | 64 | ReferenceCodeService: atomic sequential code generation (row-level DB lock) | Finance | 3 | Pilot | | 65 | Customer invoices: auto-generated for monthly rent (1st of month cron) | Finance | 3 | Pilot | | 66 | Customer invoices: auto-generated on conference hall booking approval | Finance | 3 | Pilot | | 67 | Customer invoices: manual creation by Finance | Finance | 3 | Pilot | | 68 | Invoice line items with COA account code mapping | Finance | 3 | Pilot | | 69 | LPO number field on invoices (for corporate/govt clients) | Finance | 3 | Pilot | | 70 | Invoice PDF generation (DomPDF, branded, with all 5 account payment instructions) | Finance | 3 | Pilot | | 71 | Invoice delivery via Resend email (PDF attached) + WhatsApp | Finance | 3 | Pilot | | 72 | Automated invoice reminders (3 days before, on due date, 1 day overdue, 7 days overdue) | Finance | 3 | Pilot | | 73 | Invoice status auto-update to overdue | Finance | 3 | Pilot | | 74 | Payment recording (customer receipts and vendor payments) | Finance | 3 | Pilot | | 75 | Payment method tracking (Edahab, ZAAD, bank, cheque, cash) | Finance | 3 | Pilot | | 76 | Payment auto-posts journal entry and updates account_transactions | Finance | 3 | Pilot | | 77 | Security deposit: receive, hold, and return workflow with journal entries | Finance | 3 | Pilot | | 78 | Vendor master data management | Finance | 3 | Pilot | | 79 | Purchase orders (PO) with status flow | Finance | 3 | Pilot | | 80 | Vendor bills with line items, expense account mapping, receipt upload | Finance | 3 | Pilot | | 81 | Vendor bill payment recording with journal entry | Finance | 3 | Pilot | | 82 | Expense recording with COA mapping and booking linkage | Finance | 3 | Pilot | | 83 | Expense auto-posts journal entry on save | Finance | 3 | Pilot | | 84 | Electricity rate table with effective date range history | Finance | 3 | Pilot | | 85 | Electricity meter reading entry (auto-fills previous reading) | Finance | 3 | Pilot | | 86 | Electricity charge calculation (kWh × rate, rate snapshot saved) | Finance | 3 | Pilot | | 87 | Electricity charge: add to monthly invoice OR standalone ELEC invoice | Finance | 3 | Pilot | | 88 | Journal entries: auto-posted on all financial events | Finance | 3 | Pilot | | 89 | Journal entries: manual creation by Finance for adjustments | Finance | 3 | Pilot | | 90 | General Ledger view: per-account with running balance, exportable | Finance | 3 | Pilot | | 91 | Journal view: chronological, all entries, filterable | Finance | 3 | Pilot | | 92 | Trial Balance: Σ debits = Σ credits validation | Finance | 3 | Pilot | | 93 | Balance Sheet report | Finance | 3 | Pilot | | 94 | Profit & Loss report | Finance | 3 | Pilot | | 95 | Cash Flow Statement | Finance | 3 | Pilot | | 96 | Partner Ledger (per tenant / per vendor) | Finance | 3 | Pilot | | 97 | Aged Receivables report (0-30, 31-60, 61-90, 90+ days) | Finance | 3 | Pilot | | 98 | Aged Payables report | Finance | 3 | Pilot | | 99 | Revenue Report by source | Finance | 3 | Pilot | | 100 | Expense Report by COA category | Finance | 3 | Pilot | | 101 | Occupancy Report per space per period | Finance | 3 | Pilot | | 102 | Bookings Report (filterable, exportable) | Finance | 3 | Pilot | | 103 | Payment Report (Paid/Partial/Overdue) | Finance | 3 | Pilot | | 104 | Electricity Billing Report per tenant/period | Finance | 3 | Pilot | | 105 | Payroll Summary by department/month | Finance | 3 | Pilot | | 106 | Invoice Analysis by type/status/period | Finance | 3 | Pilot | | 107 | All reports: PDF and Excel export | Finance | 3 | Pilot | | 108 | Management dashboard KPIs (revenue, tenants, bookings, overdue, account balances) | Finance | 3 | Pilot | | 109 | Revenue trend bar chart (12 months) | Finance | 3 | Pilot | | 110 | Revenue by source doughnut chart | Finance | 3 | Pilot | | 111 | Demand analytics: most-booked spaces, most popular sessions | Finance | 3 | Pilot | | 112 | Automated monthly report email (last day of month, Resend, to Admin + Super Admin) | Finance | 3 | Pilot | | 113 | Employee records (salaried and daily rate) | HR/Payroll | 3 | Pilot | | 114 | Attendance logging per employee per month | HR/Payroll | 3 | Pilot | | 115 | Leave requests (annual, sick, unpaid) with approval | HR/Payroll | 3 | Pilot | | 116 | Overtime records | HR/Payroll | 3 | Pilot | | 117 | Salary deductions (absence, advance, disciplinary) | HR/Payroll | 3 | Pilot | | 118 | Payroll run: calculate net pay per employee | HR/Payroll | 3 | Pilot | | 119 | Payslip PDF generation (branded, DomPDF, stored in S3) | HR/Payroll | 3 | Pilot | | 120 | Payslip delivery via WhatsApp + email | HR/Payroll | 3 | Pilot | | 121 | Payroll journal entry auto-posting on finalization | HR/Payroll | 3 | Pilot | | 122 | Public homepage with hero, spaces showcase, How-It-Works, CTA | Public Website | 4 | Pilot | | 123 | Browse spaces page with filter (type/floor/capacity/session) | Public Website | 4 | Pilot | | 124 | Visual floor plan SVG (color-coded availability) | Public Website | 4 | Pilot | | 125 | Individual space detail pages with gallery and pricing | Public Website | 4 | Pilot | | 126 | Catering packages display (Silver/Gold/Platinum) on space pages | Public Website | 4 | Pilot | | 127 | Online booking request form with live price summary | Public Website | 4 | Pilot | | 128 | Booking confirmation page with reference code and approval steps | Public Website | 4 | Pilot | | 129 | Waiting list signup form | Public Website | 4 | Pilot | | 130 | FAQ page (accordion) | Public Website | 4 | Pilot | | 131 | Contact page with building details | Public Website | 4 | Pilot | | 132 | Fully responsive design (mobile-first, Tailwind CSS, Poppins, brand colors) | Public Website | 4 | Pilot | | 133 | Tenant portal: login with email + password (no 2FA) | Tenant Portal | 5 | Pilot | | 134 | Tenant portal: dashboard (lease card, notifications, quick links) | Tenant Portal | 5 | Pilot | | 135 | Tenant portal: invoice list with PDF download | Tenant Portal | 5 | Pilot | | 136 | Tenant portal: electricity bills view | Tenant Portal | 5 | Pilot | | 137 | Tenant portal: conference hall bookings list with status | Tenant Portal | 5 | Pilot | | 138 | Tenant portal: documents download (signed lease, KYC uploads) | Tenant Portal | 5 | Pilot | | 139 | Tenant portal: maintenance request submission (with photo upload) | Tenant Portal | 5 | Pilot | | 140 | Tenant portal: maintenance request status tracking | Tenant Portal | 5 | Pilot | | 141 | Tenant portal: profile update and password change | Tenant Portal | 5 | Pilot | | 142 | Tenant portal: in-app notification bell (30-second polling) | Tenant Portal | 5 | Pilot | | 143 | Maintenance management: Admin assigns staff, tracks to resolution | Maintenance | 5 | Pilot | | 144 | Maintenance status

notifications (WhatsApp + email to tenant) | Maintenance | 5 | Pilot | | 145 | Broadcast announcements (Finance/Admin → tenant group → email + WhatsApp) | Communications | 5 | Pilot | | 146 | Announcement delivery log per tenant | Communications | 5 | Pilot | | 147 | Booking acknowledgement notification (WhatsApp + email on submission) | Communications | 5 | Pilot | | 148 | Booking status change notifications (all transitions, WhatsApp + email) | Communications | 5 | Pilot | | 149 | All notifications via Laravel Queue + Horizon (async, non-blocking) | Communications | 5 | Pilot | | 150 | Petty cash management (cash account, running balance, replenishment) | Finance | Future | Future | | 151 | Receipt / payment voucher PDF (auto-generated on payment receipt) | Finance | Future | Pilot | | 152 | Inter-account transfer receipt document (PDF, S3 storage) | Finance | Future | Pilot | | 153 | Budget planning vs actuals with alerts | Finance | Future | Future | | 154 | Accounts receivable aging alerts (30/60/90 day escalation) | Finance | Future | Future | | 155 | Fiscal year lock (prevent edits to closed periods) | Finance | Future | Future | | 156 | Per-event profitability (revenue – linked expenses per booking) | Finance | 3 | Pilot | | 157 | Asset register with straight-line depreciation auto-posting | Finance | Future | Future | | 158 | Credit notes against invoices | Finance | Future | Future | | 159 | Multi-currency display (USD + Somali Shilling) | Finance | Future | Future | | 160 | Recurring expense templates (auto-generate monthly) | Finance | Future | Future | | 161 | Tax / VAT configuration (configurable tax rate, currently 0%) | Finance | Future | Future | | 162 | Late payment penalties (auto-applied after grace period) | Finance | Future | Future | | 163 | Bank reconciliation tool | Finance | Future | Future |

SECTION 18 — IMPLEMENTATION NOTES FOR CLAUDE CODE CLI

This section is written directly for the developer or AI building this system. All architectural decisions are final unless explicitly changed by the client.

Core Architectural Rules

- All business logic lives in Laravel services (app/Services/) — not in controllers. Controllers are thin: validate request → call service → return response. No business logic in models or controllers.
- Service layer pattern: BookingService, InvoiceService, PaymentService, AccountingService, PayrollService, NotificationService, ReferenceCodeService. Each service is a single-responsibility class injected via Laravel's IoC container.
- AccountingService is the single entry point for all journal entry posting. Never post journal entries from controllers. All other services call AccountingService when a financial event occurs.
- ReferenceCodeService::generate(\$prefix) must use a DB transaction with SELECT FOR UPDATE to prevent duplicate codes under concurrent requests.
- Queue jobs handle: Resend email sending, WhatsApp message sending, PDF generation, auto-invoice creation, scheduled reminders. Never send emails or WhatsApp in the synchronous request cycle.
- All file uploads go to S3 via Laravel's Storage facade. Generate signed URLs for downloads (1-hour expiry). Never serve files directly from the server filesystem.
- The Next.js frontend never queries the database directly. All data via the Laravel REST API.
- Frontend auth state stored in an HTTP-only cookie managed by Sanctum. React context (AuthContext) stores the user object returned by /api/auth/me.
- All money values stored as decimal(15,2) in PostgreSQL. Never use float for currency. PHP: use string arithmetic or BCMath. Never cast to float.
- deleted_at soft-delete on all core tables EXCEPT: audit_logs, journal_entries, journal_entry_lines (these are immutable append-only records).
- CORS: Laravel config/cors.php must allow only the 3 subdomain origins. API must not be accessible from arbitrary origins.
- Rate limiting: apply Laravel's throttle middleware to all public API routes (60 req/min). Auth routes: stricter (10 req/min).

Laravel File Structure (Key Directories)

- app/Services/ — all business logic classes
- app/Services/AccountingService.php — journal entry posting for all financial events

- app/Services/BookingService.php — booking creation, status transitions, approval chain
- app/Services/InvoiceService.php — invoice creation, PDF generation, delivery
- app/Services/PaymentService.php — payment recording, invoice status update
- app/Services/PayrollService.php — payroll calculation, payslip generation
- app/Services/NotificationService.php — WhatsApp and email dispatch orchestration
- app/Services/ReferenceCodeService.php — atomic sequential code generation
- app/Jobs/ — all queue jobs (SendEmailJob, SendWhatsAppJob, GeneratePdfJob, AutoGenerateInvoicesJob, etc.)
- app/Http/Controllers/Api/ — thin controllers, one per domain (BookingController, InvoiceController, etc.)
- app/Models/ — Eloquent models with relationships and scopes. No business logic.
- resources/views/pdfs/ — DomPDF Blade templates (invoice.blade.php, payslip.blade.php, etc.)
- resources/views/emails/ — Resend email Blade templates (invoice.blade.php, booking_confirmation.blade.php, etc.)
- database/migrations/ — one file per table
- database/seeders/RoleSeeder.php — seeds Spatie roles and permissions
- database/seeders/ChartOfAccountsSeeder.php — seeds default COA
- database/seeders/SpaceSeeder.php — seeds the building's floors and spaces
- database/seeders/SystemSettingsSeeder.php — seeds default system_settings values

Next.js App Structure (Admin Dashboard)

- app/(auth)/login/page.tsx — login form
- app/(auth)/2fa/page.tsx — 2FA OTP entry
- app/(dashboard)/layout.tsx — authenticated layout with sidebar nav
- app/(dashboard)/bookings/page.tsx — booking list with tabs
- app/(dashboard)/bookings/[id]/page.tsx — booking detail
- app/(dashboard)/finance/invoices/page.tsx — invoice list
- app/(dashboard)/finance/invoices/[id]/page.tsx — invoice detail
- app/(dashboard)/finance/accounts/page.tsx — account balances
- app/(dashboard)/finance/journal/page.tsx — journal entries
- app/(dashboard)/finance/reports/[type]/page.tsx — dynamic report page
- app/(dashboard)/payroll/page.tsx — payroll runs list
- app/(dashboard)/settings/page.tsx — system settings
- lib/api.ts — Axios instance with base URL, cookie credentials, CSRF handling
- lib/auth.ts — auth context, useAuth hook, permission check
- hooks/usePermission.ts — returns true/false for a given permission string
- components/ui/ — shared UI components (Button, Badge, Modal, Table, etc.)

Key API Endpoints Reference

Method	Endpoint	Description
POST	/api/auth/login	Email + password → returns challenge token if 2FA needed
POST	/api/auth/2fa/verify	Verify OTP → issues Sanctum cookie
POST	/api/auth/logout	Destroys session
GET	/api/auth/me	Returns authenticated user with role and permissions array
GET	/api/bookings	List bookings (with filters: status, product_id, date range)
POST	/api/bookings	Create booking (public or staff)
GET	/api/bookings/{id}	Booking detail
PATCH	/api/bookings/{id}/status	Status transition (approve/reject/cancel)
GET	/api/products	List products (with filters)
POST	/api/products	Create product
PUT	/api/products/{id}	Update product
GET	/api/invoices	List invoices
POST	/api/invoices	Create invoice
POST	/api/invoices/{id}/send	Send invoice to tenant
GET	/api/invoices/{id}/pdf	Download invoice PDF (signed URL)
POST	/api/payments	Record a payment
GET	/api/accounts	List accounts with computed balances
POST	/api/accounts/transfer	Inter-account transfer
GET	/api/accounting/trial-balance	Trial balance data
GET	/api/accounting/ledger	General ledger (filtered by account + date)
GET	/api/reports/{type}	Financial report data (type =

balance-sheet, pl, cash-flow, etc.) | | GET | /api/electricity/readings | List electricity readings | | POST | /api/electricity/readings | Record new reading | | GET | /api/payroll/runs | List payroll runs | | POST | /api/payroll/runs | Create payroll run | | POST | /api/payroll/runs/{id}/finalize | Finalize run and send payslips | | GET | /api/chart-of-accounts | List COA accounts | | GET | /api/settings | Get all settings | | PUT | /api/settings | Update settings (batch) | | GET | /api/audit-logs | List audit logs (Admin + Super Admin only) | | GET | /api/tenant/dashboard | Tenant portal: dashboard data (scoped to auth tenant) | | GET | /api/tenant/invoices | Tenant portal: tenant's own invoices | | POST | /api/tenant/maintenance | Tenant portal: submit maintenance request | | GET | /api/tenant/notifications | Tenant portal: notifications | | PATCH | /api/tenant/notifications/{id}/read | Mark notification as read |

Scheduled Commands (Laravel Scheduler)

| Command | Schedule | Purpose | | --- | --- | --- | | AutoGenerateMonthlyInvoices | Monthly, 1st at 07:00 | Creates invoices for all active monthly leases | | LeaseExpiryReminder | Daily at 08:00 | Sends renewal reminders for leases expiring in ≤ 10 days | | InvoiceReminderCheck | Daily at 09:00 | Sends reminders for invoices due in 3 days, due today, 1 day overdue, 7 days overdue | | WaitingListFollowUp | Daily at 10:00 | Escalates waiting list notifications if previous contact did not respond within 24 hours | | GenerateMonthlyReport | Last day of month at 23:30 | Compiles and emails monthly summary to Admin + Super Admin | | UpdateLeaseStatuses | Daily at 00:05 | Sets lease status to expired where end_date < today | | PruneAuditLogs | Never (audit logs are permanent) | N/A — do not implement log pruning |

WhatsApp Message Templates

WhatsApp Business API requires pre-approved message templates for outbound messages. The following templates must be submitted for approval before launch:

- booking_acknowledgement — 'Hello {name}, your booking request for {space} on {date} ({session}) has been received. Reference: {booking_code}. We will confirm within 24 hours.'
- booking_approved — 'Great news {name}! Your booking {booking_code} for {space} on {date} ({session}) is confirmed. Total: \${amount}. Invoice attached.'
- booking_rejected — 'Hello {name}, your booking {booking_code} for {space} on {date} has not been approved. Reason: {reason}. Contact us to discuss alternatives.'
- invoice_reminder — 'Hello {name}, invoice {invoice_code} for \${amount} is due on {due_date}. Pay via Edahab/ZAAD/Bank using your invoice code. View: {portal_link}'
- invoice_overdue — 'Hello {name}, invoice {invoice_code} for \${amount} is now overdue. Please make payment immediately to avoid disruption. Contact: {phone}'
- lease_renewal — 'Hello {name}, your lease for {space} expires on {end_date}. Please contact us to discuss renewal. {contact_email}'
- maintenance_update — 'Hello {name}, your maintenance request {request_code} has been updated to: {status}. Notes: {notes}'
- payslip — 'Hello {name}, your payslip for {month} is ready. Net pay: \${net_pay}. Download from your email or contact HR.'
- waiting_list_slot — 'Hello {name}, a slot has opened for {space} on {date} ({session}). Reply YES to secure your place within 24 hours.'

Testing Checklist (Pre-Launch)

- 1. Auth: all 4 staff roles can log in, 2FA OTP delivered and validates correctly, lockout after 5 failed attempts
- 2. RBAC: each role can only access its permitted routes — verify forbidden routes return 403
- 3. Booking flow: submit via public form → appears in admin_pending → Admin approves → Finance approves → booking_approved → invoice auto-created → client notified

- 4. Booking rejection: reject at Admin stage → rejection notification sent → reject at Finance stage → rejection notification sent
- 5. Invoice: auto-generated on 1st of month for active lease — verify invoice_code, due_date, line items
- 6. Invoice PDF: download, verify branding, line items, total, payment instructions
- 7. Payment recording: record full payment → invoice status = paid → journal entry posted → account balance updated
- 8. Journal entry balance: after a sequence of transactions, verify trial balance (Σ debits = Σ credits)
- 9. Electricity: record reading → calculate charge → add to invoice → status = invoiced
- 10. Payroll: create payroll run → calculate net pay → finalize → payslip PDFs in S3 → WhatsApp/email delivered
- 11. Tenant portal: log in as tenant → view invoices → download PDF → submit maintenance request
- 12. Public website: submit booking form → confirmation page shows BK code → booking visible in admin
- 13. Waiting list: book a fully-booked slot → waiting list entry created → cancel approved booking → notification sent to first in queue
- 14. Audit log: perform 5 state-changing actions → verify all 5 appear in audit_logs with correct fields
- 15. WhatsApp: all 9 templates send correctly from all triggers
- 16. Resend email: all templates send correctly, PDF attachments deliver
- 17. S3: all uploads succeed, signed URLs work and expire correctly
- 18. Scheduled jobs: test AutoGenerateMonthlyInvoices and LeaseExpiryReminder manually via php artisan schedule:run
- 19. Queue: verify Horizon shows no failed jobs after running test scenarios
- 20. Performance: dashboard KPIs load in <2 seconds on a clean cache

SECTION 19 — v3.1 ADMIN PANEL ENHANCEMENTS

Version 3.1 incorporates a round of admin-experience enhancements layered on top of the completed Phase 1–3 build, plus the production deployment runbook for the admin dashboard. These items refine usability and operational speed; they do not change the data model or financial logic established in v3.0.

19.1 Cross-Cutting UX Enhancements

- **Staff Notification Bell** — A header bell with a live unread count. Aggregates everything that needs the signed-in user's attention, computed on demand (always accurate, never stale): bookings awaiting Admin approval, bookings awaiting Finance approval, leases awaiting approval, pending leave requests, overdue invoices, leases expiring within 10 days, and tenant documents expiring within 30 days. Polls every 60 seconds. Clicking an item navigates straight to the record. Role-aware — each user only sees alerts they can act on.
- **Approvals Inbox** — A dedicated `/dashboard/inbox` screen consolidating every actionable item for the signed-in user into one prioritised list with inline action buttons (approve / reject / view). Replaces hunting across separate list pages.

- **Global Command Palette (Ctrl/⌘ + K)** — A keyboard-driven quick switcher available on every screen. Searches across bookings (BK-), invoices (INV-), vendor bills (VB-), tenants, employees, and vendors by code or name, and offers instant navigation to any admin page. Backed by a single `GET /api/search?q=` endpoint that respects the user's permissions.
- **Profile Photo Upload** — Staff can upload an avatar from the profile slide-over; stored via the Storage facade and surfaced on the header and user lists (`users.profile_photo_url`).
- **Document Expiry Tracking** — `tenant_documents` gains an optional `expiry_date` . Documents nearing expiry (≤ 30 days) are surfaced in the notification bell and on the dashboard, so KYC papers, business registrations, and notarised contracts are renewed before they lapse.

19.2 Tabbed Management Dashboard

The dashboard is restructured from a single long scroll into a **tabbed, no-scroll layout**, each tab fitting one laptop viewport:

- **Overview tab** — slim KPI strip (staff, tenants, leases, pending bookings, upcoming, overdue), the user's Action Center (approvals/alerts with inline buttons), and account-balance summary.
- **Finance tab** — revenue this month / YTD, outstanding AR, overdue total, account balances, revenue-trend bar chart, and revenue-by-source doughnut (Finance / Admin / Super Admin only).
- **Operations tab** — booking pipeline by status, upcoming bookings, waiting list, expiring leases, top-booked spaces and popular sessions. Tab visibility is role-aware; cards use fixed heights and denser spacing so no tab requires scrolling on a standard screen.

19.3 Enhancement Feature Register (v3.1)

| # | Feature | Area | Status | | --- | --- | --- | --- | | E1 | Staff notification bell with live counts (role-aware) | UX | Pilot | | E2 | Unified Approvals Inbox (`/dashboard/inbox`) | UX | Pilot | | E3 | Global command palette (Ctrl/⌘+K) + `/api/search` | UX | Pilot | | E4 | Profile photo upload | Users | Pilot | | E5 | Tenant document expiry date + expiry alerts | Tenants | Pilot | | E6 | Tabbed dashboard (Overview / Finance / Operations) | Dashboard | Pilot | | E7 | Create operating account (auto-creates linked Chart-of-Accounts code) | Accounting | Pilot |

19.4 Operating Account Creation (E7)

Phase 3 seeded a fixed set of operating accounts (Petty Cash, Edahab/ZAAD lines, Darasalam Bank). This enhancement lets **Super Admin & Admin** add new operating accounts at runtime from **Accounting → Operating Accounts → “+ New Account”**.

- **Inputs:** name, type (`cash` / `bank` / `mobile_money`), optional account identifier, optional notes.
- **Automatic Chart-of-Accounts link:** on creation the system creates a matching **asset** COA entry and links it, so the account immediately works with journals, transfers and the trial balance. Codes are auto-assigned from the type's range (flat, matching the seed):
 - Cash → `1001-1009`
 - Mobile Money → `1010-1019`
 - Bank → `1020-1029`
- **Opening balance:** none — accounts start at `$0.00` ; balances accrue from real transactions only (keeps the ledger balanced with no opening journal).
- **Access:** restricted to `super_admin` and `admin` (route role middleware); the created COA code is **non-system** so it remains editable/removable. Creation is written to the audit log.

SECTION 20 — ADMIN PANEL DEPLOYMENT RUNBOOK (VPS)

Production deployment of the admin dashboard (`admin.halelotower.so`) and the Laravel API (`api.halelotower.so`) onto the client VPS (Ubuntu 22.04). This is the concrete, step-by-step procedure referenced in Section 12.

20.1 Server Prerequisites

- Ubuntu 22.04 LTS VPS with a public IP and root/sudo SSH access
- DNS A-records: `admin.halelotower.so` and `api.halelotower.so` → the VPS IP
- Packages: PHP 8.3 + extensions (`php8.3-fpm` `php8.3-pgsql` `php8.3-mbstring` `php8.3-xml` `php8.3-bcmath` `php8.3-curl` `php8.3-zip` `php8.3-gd`), Composer, Node.js 20 + npm, PostgreSQL 16, Redis 7, Nginx, Certbot, Supervisor, Git

20.2 Database

```
sudo -u postgres psql
CREATE DATABASE haleelo_tower;
CREATE USER haleelo_user WITH ENCRYPTED PASSWORD '<strong-password>';
GRANT ALL PRIVILEGES ON DATABASE haleelo_tower TO haleelo_user;
\q
```

20.3 API (Laravel) — `/var/www/api`

```
cd /var/www && git clone <repo> api && cd api
composer install --no-dev --optimize-autoloader
cp .env.example .env # then edit (see 20.6)
php artisan key:generate
php artisan migrate --force
php artisan db:seed --force # RoleSeeder, SystemSettingsSeeder, ChartOfAccounts, Accounts, Floors, Spaces
php artisan storage:link
php artisan config:cache && php artisan route:cache && php artisan event:cache
sudo chown -R www-data:www-data storage bootstrap/cache
```

20.4 Admin (Next.js) — `/var/www/admin`

```
cd /var/www && git clone <repo> admin && cd admin
npm ci
# set NEXT_PUBLIC_API_URL=https://api.halelotower.so in .env.local
npm run build
pm2 start "npm run start" --name haleelo-admin # binds to port 3000
pm2 save && pm2 startup
```

20.5 Nginx + SSL

- Two server blocks: `api.halelotower.so` → PHP-FPM (Laravel `public/`), `admin.halelotower.so` → reverse proxy to `127.0.0.1:3000` .
- `sudo certbot --nginx -d admin.halelotower.so -d api.halelotower.so` (auto-renew enabled).
- The committed `deploy/` folder contains ready Nginx vhosts, the Supervisor Horizon config, the PM2 ecosystem file, and a `runbook.md` with the full command sequence.

20.6 Production .env (API) — key differences from dev

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://api.halelotower.so
ADMIN_URL=https://admin.halelotower.so
SESSION_DOMAIN=.halelotower.so
SESSION_SECURE_COOKIES=true
SANCTUM_STATEFUL_DOMAINS=admin.halelotower.so
FILESYSYSTEM_DISK=s3 # set AWS_* credentials, or keep 'public' if not using S3 yet
QUEUE_CONNECTION=redis
CACHE_DRIVER=redis
RESEND_API_KEY=<key> # required for email delivery
WHATSAPP_* =<credentials> # required for WhatsApp delivery
```

20.7 Background workers & cron

- **Supervisor** runs Laravel Horizon (queue worker) — config in `deploy/horizon.conf`.
- **Cron** — single entry: `* * * * * cd /var/www/api && php artisan schedule:run >> /dev/null 2>&1` (drives lease expiry, monthly invoices, reminders, monthly report).

20.8 Post-deploy smoke test

1. `https://admin.halelotower.so` loads; Super Admin logs in (2FA OTP delivered).
2. Create a booking → approve chain → invoice auto-created.
3. Record a payment → trial balance balanced.
4. Horizon dashboard shows no failed jobs; `php artisan schedule:list` shows all scheduled commands.

SECTION 21 — POST-PILOT IMPROVEMENT BACKLOG

(Manager & Accountant review, June 2026. Status: Planned — added under v3.2 without a version bump per client instruction. Each item below carries the design decisions confirmed during planning.)

21.0 Backlog Register

| # | Improvement | Area | Access | Status | | --- | --- | --- | --- | --- | | B1 | Edit + activate/deactivate operating accounts | Accounting | Super Admin, Admin | Planned | | B2 | Purchase Order PDF (vendor-facing) | Procurement | per view-perms | Planned | | B3 | Invoice resend | Finance | manage-invoices/send-invoice | Planned | | B4 | Document delivery via WhatsApp + per-category channel settings | Comms | Super Admin (settings) | Planned | | B5 | Staff coupon codes (booking discounts) | Bookings/Finance | Super Admin manages | Planned | | B6 | Trial Balance — date range, opening/closing, comparison | Accounting | view-financial-reports | Planned | | B7 | Manual Journal — auto-balance helper | Accounting | create-journal-entries | Planned | | B8 | Auto account reconciliation (statement upload) | Accounting | manage-accounts | Planned | | B9 | Animated calendar + click/drag to create booking | Bookings | view/create-booking | Planned |

21.1 Operating Account Management (B1)

Building on §19.4 (account creation).

- **Edit:** Super Admin & Admin can edit an existing operating account's **name, account identifier (number), and notes**. The linked **Chart-of-Accounts code is immutable** (never editable from here).
- **Active/Inactive:** accounts carry an `active` flag. **Only active accounts appear in selection dropdowns** (payments, transfers, expenses, electricity, payroll payout). The Accounts page lists all accounts with a status badge + active/inactive filter and a toggle.
- **Deactivation with a balance:** if the account's computed balance $\neq 0$, deactivation **prompts for a destination account** and posts a **balanced inter-account transfer journal** (debit destination / credit this account) to zero it, then marks it inactive — so the ledger stays balanced. Zero-balance accounts deactivate directly. Reactivation simply re-enables selection.
- **Access:** `role:super_admin|admin`. All edits, (de)activations and sweep-transfers are audit-logged.

21.2 Purchase Order PDF (B2)

- Branded, printable PO PDF mirroring the invoice PDF: building header, **vendor details**, PO code & date, expected delivery date, line items (description, qty, unit price, line total), subtotal/total, and notes/terms.
- Endpoint `GET /purchase-orders/{po}/pdf`; **Export PDF** button on the PO detail page. Intended to be printed/handed or sent to the vendor (see B4).

21.3 Invoice Resend (B3)

- A **Resend** action re-delivers an already-issued invoice through the configured channel(s) (see B4). It **does not re-post the AR journal** — only the first send posts accounting. Tracks `last_sent_at` and a resend counter; audit-logged. Available for invoices in `sent / partial / overdue`.

21.4 Document Delivery via WhatsApp + Channel Settings (B4)

- Send documents over WhatsApp using the existing Twilio WhatsApp integration. PDFs are delivered as a **media message using the document's signed S3 URL** (private bucket; the signed link outlives delivery).
- **Sendable documents:** Invoices (issued + overdue reminders), Purchase Orders (to vendors), Payment receipts, Booking confirmations, and general notices/announcements.
- **Settings — per-category channel preference** (new keys): `comm_channel_invoicing`, `comm_channel_receipts`, `comm_channel_notices` — each $\in \{ \text{email}, \text{whatsapp}, \text{both} \}$. The relevant send/resend flows fan out to the selected channel(s).
- **Templates:** business-initiated WhatsApp messages require **Meta-approved templates**. The client creates the templates in Meta Business Manager using §21.4.1; template *names* are referenced from config so code stays stable.

21.4.1 WhatsApp Templates to create in Meta Business Manager

Create each as language **English**, category **UTILITY** (except `general_notice` → **MARKETING** if used for promotions). Templates that carry a PDF use a **Header → Document**; the body variables are positional `{{n}}`.

- **invoice_issued** — Header: Document (the invoice PDF). Body: Hello `{{1}}`, your invoice `{{2}}` from Haleelo Tower is ready. Amount due: `{{3}}`, due by `{{4}}`. Thank you for your business. (1 = client name, 2 = invoice code, 3 = amount, 4 = due date)
- **invoice_overdue** — Header: none. Body: Reminder: invoice `{{1}}` for `{{2}}` is overdue. Outstanding balance: `{{3}}` (was due `{{4}}`). Please arrange payment at your earliest convenience. — Haleelo Tower (1 = invoice code, 2 = client/period, 3 = balance, 4 = due date)
- **purchase_order** — Header: Document (the PO PDF). Body: Hello `{{1}}`, Haleelo Tower has issued Purchase Order `{{2}}` dated `{{3}}`, total `{{4}}`. Please find the PO attached. (1 = vendor name, 2 = PO code, 3 = date, 4 = total)
- **payment_receipt** — Header: Document (the voucher PDF). Body: Thank you `{{1}}`. We've received your payment of `{{2}}` on `{{3}}` (ref `{{4}}`). A receipt is attached. — Haleelo Tower (1 = payer name, 2 = amount, 3 = date, 4 = reference)
- **booking_confirmation** — Header: Document (booking confirmation PDF). Body: Hi `{{1}}`, your booking `{{2}}` at Haleelo Tower is confirmed for `{{3}}` (`{{4}}`) in `{{5}}`. Total: `{{6}}`. (1 = client, 2 = booking code, 3 = date, 4 = session, 5 = space, 6 = total)
- **general_notice** — Header: none. Body: Haleelo Tower notice — `{{1}}`: `{{2}}` (1 = subject, 2 = message)

21.5 Staff Coupon Codes — Booking Discounts (B5)

- Each **staff login user** has a unique **coupon code** with a **fixed discount %** set within their **role cap**: Admin $\leq 50\%$, Operations $\leq 25\%$, Finance $\leq 30\%$. **Super Admin** can set any % and manages (create/edit/disable/regenerate) all codes.
- **Scope: bookings only.** Applying a valid, active code during booking creation discounts that booking's invoice.
- **Accounting — contra-revenue method.** The invoice records **gross revenue** at full price; the discount posts as a separate debit to a new contra-revenue account **3090 - Sales Discounts** (a revenue-type, debit-normal account that nets down revenue on the P&L). Example for a \$100 booking at 20% off:
 - Dr Accounts Receivable **\$80**, Dr Sales Discounts (3090) **\$20**, Cr Revenue **\$100** — debits = credits = \$100. The customer owes only the **net \$80**; on payment, Dr Cash \$80 / Cr AR \$80.
 - This keeps the books balanced at every step **and** makes total discounts given reportable (overall and, via the stored coupon/owner, per employee). No accounting-engine change is needed — **3090** is just a revenue account carrying debits.

- The applied coupon and its owning user are recorded on the booking/invoice for attribution. Validation enforces the owner's role cap server-side; disabled/expired codes are rejected. All applications are audit-logged.
- **COA addition:** seed `3090 - Sales Discounts` (type `revenue`, system account).

21.6 Trial Balance — Date Range, Opening/Closing & Comparison (B6)

- **From/To** date pickers. For the selected range, each account shows: **opening balance** (cumulative before *From*), **period movement** (debits/credits within range), and **closing balance** (as of *To*). Column totals must balance.
- **Comparison mode:** two independent ranges (e.g., **May vs June, Q1 2025 vs Q1 2026**) shown side-by-side with **variance** (amount and %). Exportable to PDF/Excel.

21.7 Manual Journal Entry — Auto-Balance Helper (B7)

- In the manual journal form, typing an amount on one side **auto-fills the opposite side** so the entry balances (the engine already rejects unbalanced entries). For a two-line entry the counter-line mirrors the amount; for multi-line entries the remaining empty line is filled with the running difference. Server-side balance validation is unchanged.

21.8 Auto Account Reconciliation (B8)

- For a chosen account, the user **uploads its statement** (ZAAD / Edahab / bank) as **CSV or Excel** (PDF supported on a **best-effort** basis — layouts vary).
- The system matches statement rows to recorded `AccountTransactions` by **amount + date ($\pm$ tolerance) + reference**, then presents three buckets: **matched** (marked reconciled), **in statement but not in books** (offer to create the missing entry), and **in books but not on statement** (flag for review). A summary compares **statement closing balance vs book balance**.
- Per-provider **column mapping** is configurable. CSV/Excel are strongly preferred for accuracy.

21.9 Calendar Enhancements (B9)

- Polished, **animated** calendar (navy/gold theme, smooth transitions, event hover/preview, status-color legend).
- **Create from calendar:** single-click a day opens the new-booking form with the **date prefilled**; **click-drag** a time range prefills date + **start/end time**, then opens the form.

END OF DOCUMENT
